FEDERAL URDU UNIVERSITY OF ARTS, SCIENCE AND TECHNOLOGY

Final Year Project (Phase-I)

Project Documentation

Submitted by:

1. Rida Eman
2. Muhammad Mahhin Shahzad

Supervised By:

Mr. Naeem Akhter Malik

Lecturer, Department of Computer Science

Date: 07-01-2026

Project Proposal

Of

Smart Transaction Sorter & Analytics System

Group Members:

1. Rida Eman
2. Mahhin Shahzad

Project Supervisor:

Mr:Naeem Akhter Malik

Supervisor's Section

Supervisor Comments:	**Yes**	**No**
• Proposal Idea is good enough to take it as a Final Project.	☐	☐
• Are they doing Project for the Real Client, if yes, Students must provide evidence letter in Proposal	☐	☐

Supervisor Comments (if any): ______________________________

____

❖ Project Proposal is accepted under your Supervision.	☐	☐

Supervisor Signature with Date:

____

Project (P1) Instructor (BS-7A) Eve

Mr:Naeem Akhter Malik

❖ Project Proposal is approved. Submission Date: ___________	☐	☐

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

The rise of the digital economy in Pakistan has been driven by the widespread adoption of fintech platforms such as Easypaisa, SadaPay, and JazzCash. These services have empowered millions of users by simplifying payments and providing instant access to their complete transaction history. This readily available data holds the potential to offer valuable insights into personal spending habits. However, for most users, this potential remains untapped as the data is typically provided in raw formats (e.g., CSV files) without sophisticated analytical tools.

This report documents the Smart Transaction Sorter and Analytics system, a web-based application designed to unlock the value hidden within this raw financial data. The project's academic objective is to develop a functional, standalone tool that serves as a research-based feasibility study. The resulting application is structured as a practical blueprint for an intelligent analytics module. The long-term vision, while outside the immediate scope of this project, is that such a module could be pitched for integration into the very fintech platforms it complements, thereby closing the significant analytical gap in the current market.

1.2 Problem Statement

The problem is the absence of an integrated Business Intelligence (BI) and analytics layer in Pakistan digital wallets. People export their data, move it into another tool, and tag each transaction by hand. It is a manual and time-consuming process.

1.3 Proposed System

The proposed system is a standalone, web-based application focused on delivering a complete workflow from raw CSV data to a finalized BI dashboard. To automate data preparation for the dashboard, an intelligent sorting component using Gemini API is included. The system aims to demonstrate feasibility using realistic transaction data while remaining independent from direct banking or wallet API integrations. The core functionality is broken down as follows:

1. **Custom Category Management:** The user will first define their own set of personal spending categories (e.g., Groceries, Transport, Health & Fitness).
2. **CSV Data Ingestion:** The user then uploads their transaction history as a standard CSV file. This serves as the primary data input mechanism.
3. **Intelligent Sorting Component:** A smart classification component, utilizing Gemini API, processes the raw data. It analyzes transaction descriptions and human-like notes/comments, mapping each entry to the most relevant category from the user's own custom list. This automated step moves beyond simple rule-based mapping to understand context, preparing the data accurately for the BI layer.

4. **Interactive BI Visualization:** Once sorted, the data is presented to the user through an interactive Business Intelligence (BI) dashboard. This dashboard is the primary output and uses Metabase to visualize spending breakdowns tailored to the user's personal financial structure.
5. **Validation with Public Data:** To demonstrate the effectiveness and real-world applicability of the sorting component, the system will be tested and validated using a relevant public dataset. This dataset will contain transaction-like entries with descriptions and comments, allowing for rigorous evaluation of the classification accuracy without relying on proprietary user data.

By leveraging a sorting mechanism powered by Gemini API, adaptable to user categories and validated on realistic transaction data, the application delivers personalized and accurate insights through its core BI and visualization dashboard.

1.4 Survey of the Related Applications

To establish the necessity of this project, this survey analyzes two distinct application categories: the local digital wallets that create the data and the problem, and the global expense trackers that represent existing but flawed solutions.

1.4.1 A. Market Survey — Local Digital Wallets (Direct Context)

A review of the official documentation for Pakistan's leading digital wallets confirms that while they excel at payments, they generally lack a mature, integrated analytics tool.

- **SadaPay, NayaPay, & Easypaisa:** These platforms reliably provide users with the ability to download their account statements or view transaction histories, with some confirming CSV export availability. However, their feature sets do not include built-in BI or smart analytics layers.
- **JazzCash:** While also providing standard statements, JazzCash recently announced a Spending Tracker feature. Based on its public introduction, this appears to be a basic tracking tool for monitoring budgets. While a positive step, it is distinct from a full-scale BI layer and does not offer the intelligent, personalized categorization that is central to this proposal.

Implication. The local market either offers no native analytics or, at best, provides basic tracking. This creates a clear analytics vacuum and validates the need for a separate application to provide the missing intelligence.

1.4.2 B. Indirect Comparators — Global CSV-Import Tools

Global platforms like YNAB and Mint demonstrate the feasibility of the CSV-import workflow but are poorly suited to the local context and fail to solve the core problem of personalization.

- **High-Friction Workflow:** Tools like YNAB and Mint, while powerful, still require significant manual effort after a CSV import to clean up data and assign categories line-by-line. This does not solve the primary issue of laborious manual work.

- **Lack of Personalization and Intelligence:** These platforms are separate, often paid services that use rigid, generic categorization systems. They lack the intelligent sorting component needed to learn and adapt to a user's own custom financial categories, failing the key protein bar test case.
- **Misaligned Business Model:** Their primary model is direct bank API integration, which is not the standard in the Pakistani market. Their CSV import tools are a secondary feature, not the core product.

Implication. The global tools prove the CSV-to-analytics model is viable but highlight a gap for a more intelligent, low-friction, and personalized solution tailored to the realities of the local market. This project is designed specifically to fill that gap.

1.5 Modules & Sub-modules

The system architecture will be broken down into the following modules and sub-modules to ensure a clear separation of concerns and facilitate iterative development.

1.5.1 User & Category Management Module

This module will handle all user-facing administrative tasks and personalization features.

- **Authentication Submodule:** This submodule is responsible for secure user registration, login, logout, and session management to ensure data privacy.
- **Category CRUD Submodule:** This provides the interface and backend logic for users to Create, Read, Update, and Delete (CRUD) their personalized list of spending categories.

1.5.2 Data Ingestion Module

This module serves as the primary data entry point for transaction histories.

- **File Upload Submodule:** This submodule provides the secure, user-facing web form for uploading transaction data in CSV format.
- **CSV Validation Submodule:** This is a backend service that parses the uploaded file and verifies its structural integrity, ensuring required columns (e.g., Date, Description, Amount) are present.

1.5.3 Intelligent Sorting Module

This module contains the core data processing and classification logic.

- **Data Pre-processing Submodule:** This submodule cleans and standardizes the validated data, converting data types (e.g., string dates to date objects) to prepare it for classification.

- **Classification Engine Submodule:** This is the smart component that takes a transaction's text and the user's custom category list as input, and outputs the most probable category.
- **Database Persistence Submodule:** This submodule saves the processed transaction data, along with its newly assigned category, into the database.

1.5.4 BI & Visualization Module

This module is responsible for presenting the final analytical output to the user.

- **Token Authentication Submodule:** This is a backend service that compiles secure, signed JSON Web Tokens containing user session parameters using the HMAC-SHA256 algorithm.
- **Dashboard Rendering Submodule:** This is the frontend component that renders the securely embedded Metabase dashboard inside an iframe, applying the token parameters to display user-scoped charts and statistics.

1.6 Primary Actors

The system is designed for a single primary actor:

- **The User:** An individual who wants to analyze their personal financial transaction history. The User will be able to perform the following actions:
  - Register for an account and log in securely.
  - Create and manage their own personalized list of spending categories.
  - Upload their transaction history in CSV format.
  - View the interactive BI dashboard with visualizations of their spending.
  - (Optional) Review and manually correct any automated classifications.

1.7 Tools & Techniques

The project uses a modern, stable technology stack to ensure scalability and maintainability.

- **Backend Framework:** Python with the Django Framework is used to build the server-side logic, manage URLs, and handle database interactions.
- **Database:** PostgreSQL is the confirmed database target for the final system because it supports reliable relational storage, indexing, and production deployment.
- **Data Processing:** Python CSV handling, Django forms, and model validation are used for efficient parsing, validation, and storage of uploaded transaction data.
- **Intelligent Sorting:** Gemini API is used to implement the classification component through structured prompts, JSON responses, confidence scoring, and category validation.

- **Frontend Visualization:** Metabase is used as the Business Intelligence layer and is embedded into the Django interface through a secure dashboard URL.
- **Web Technologies:** The user interface is built with Django templates, HTML, CSS, JavaScript, Bootstrap, and Jazzmin for the administrative interface.

1.8 Process Model & Design Approach

The project will be developed using an Iterative and Incremental process model. This approach is well-suited for a modular system, as it allows for the development and testing of each component in successive cycles or iterations.

The development will proceed by building and testing each module individually before integrating them into a cohesive whole. This allows for early feedback, easier debugging, and a more manageable workflow. The design approach will follow the Model-View-Template (MVT) architectural pattern, which is native to the Django framework and provides a clear separation between data logic, business logic, and user interface.

1.9 Project Plan

The project timeline is scheduled over approximately seven months (30 weeks), visualized in the Gantt chart in Figure 1.1 below. The plan prioritizes documentation and core feature development first, followed by the integration of the intelligent sorting component and finalization.

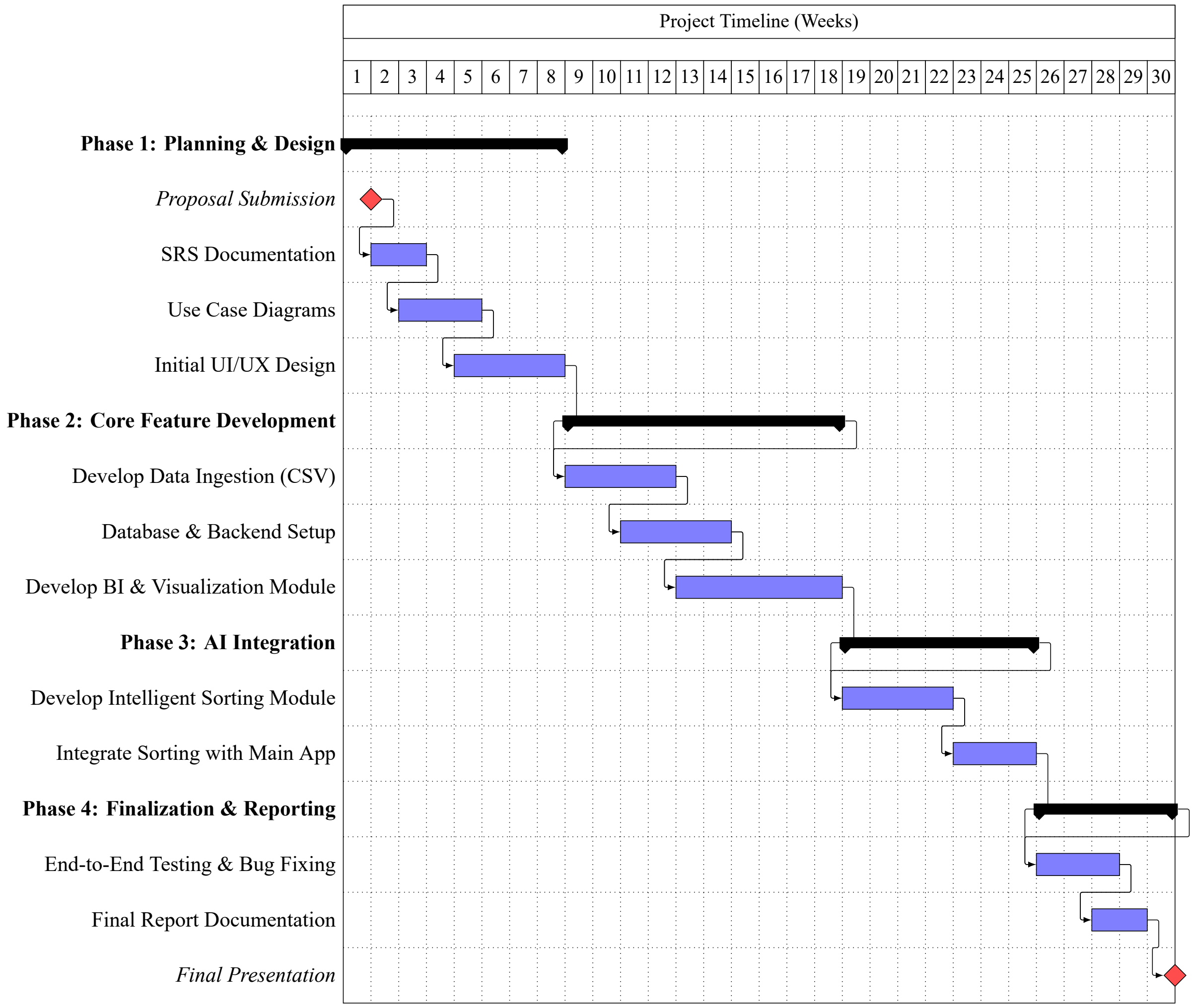

Figure 1.1: Project development timeline presented as a Gantt chart.

Chapter 2

Requirements Analysis

2.1 Purpose

The purpose of the Smart Transaction Sorter and Analytics System is to provide users with an intelligent, automated way to analyze their personal financial spending. This web application uses Artificial Intelligence (AI) to categorize raw transaction data (from CSV files) and instantly convert it into clear, visual charts, eliminating the need for hours of frustrating manual work.

2.2 Project Scope

The scope of this project is a standalone, web-based tool that accepts a financial history file (CSV), uses AI to categorize every line based on user-defined labels, and presents the results on a BI Dashboard. The system includes User authentication, Category Management, CSV Upload/Validation, AI Sorting Logic, Data Persistence, and a Visualization Dashboard.

- **In Scope:** User login, Category creation, CSV parsing, AI classification, Dashboard charts.
- **Out of Scope:** Direct integration (API) with bank/wallet accounts, payment processing, or real-time transaction tracking.

2.3 Definitions, Acronyms, and Abbreviations

Term/Abbreviation	Definition
STS	Smart Transaction Sorter (The name of this application).
AI	Artificial Intelligence (The component that performs automatic classification).
CSV	Comma-Separated Values (The standard file format for data input/upload).
BI	Business Intelligence (Refers to the analytical dashboard and visualization tools).
DB	Database (The system storage where user and financial data are saved).

2.4 Document Overview

This document describes the requirements for the Smart Transaction Sorter and Analytics System (STS). It serves as the single reference point for what the system must do and how well it must perform. The STS acts as a complimentary Business Intelligence layer for existing digital wallets and bank applications that currently only offer raw data exports.

2.5 Functional Requirements

2.5.1 Security Management

Process Sign Up

ID	Requirement Description
SRS-1	The system shall allow new users to securely register by providing a username, email, and password.
SRS-2	The system must validate that the email address is unique and not already registered.
SRS-3	The system shall store user passwords using a strong one-way hash (never in plain text).
SRS-4	Upon successful registration, the system shall create a dedicated, isolated database scope for the user.

Process Sign In / Logout

ID	Requirement Description
SRS-5	The system shall allow registered users to log in securely using their credentials.
SRS-6	The system shall use HTTPS/SSL for all data transfer between the client and server to prevent interception.
SRS-7	The system shall allow users to securely log out, terminating their current session.

2.5.2 Category Management

Create Spending Category

ID	Requirement Description
SRS-8	The system shall allow users to create new, personalized spending category labels (e.g., ”Groceries”, ”Utilities”).
SRS-9	The system shall validate that the category name is not a duplicate within the user’s profile.

SRS-10	The system shall ensure all created categories are immediately available for the AI Sorting Engine to use.

Manage Spending Categories

ID	Requirement Description
SRS-11	The system shall allow users to view a list of all their existing categories.
SRS-12	The system shall allow users to edit the name of an existing category.
SRS-13	The system shall allow users to delete a category (and optionally reassign its associated transactions).

2.5.3 Data Ingestion Management

Upload Transaction File (CSV)

ID	Requirement Description
SRS-14	The system shall provide a simple interface for the user to upload a transaction history file.
SRS-15	The system is constrained to accepting only CSV (Comma-Separated Values) file formats.
SRS-16	The system shall provide a progress indicator or status feedback during the upload process.

File Validation

ID	Requirement Description
SRS-17	The system shall check the uploaded file for necessary columns (specifically: Date, Description, and Amount).
SRS-18	The system shall provide clear, user-friendly error messages if the file structure is invalid or missing data.
SRS-19	The system shall reject files that exceed a predefined size limit to maintain performance.

2.5.4 Intelligent Sorting Management

Process Transactions (AI Sorting)

ID	Requirement Description
SRS-20	The system shall use a smart classification model (AI) to read transaction descriptions from the uploaded file.
SRS-21	The system shall automatically assign the most relevant custom category to each transaction.
SRS-22	The system shall utilize Gemini API for the classification logic.

Data Persistence & Saving

ID	Requirement Description
SRS-23	The system shall securely save the processed transaction data, including the new category label, to the user's database.
SRS-24	The system shall ensure that once categorized, the financial history persists until explicitly deleted by the user.

2.5.5 Analytics Management

View Dashboard

ID	Requirement Description
SRS-25	The system shall display a primary BI (Business Intelligence) dashboard view after processing is complete.
SRS-26	The dashboard shall be interactive, allowing users to hover over elements for more details.

View Spending Breakdown

ID	Requirement Description
SRS-27	The system shall display clear charts (e.g., pie charts) visualizing total spending broken down by custom categories.

SRS-28	The system shall calculate and display total expenditure for the uploaded period.

2.5.6 Administrator Management

Admin Panel Access

ID	Requirement Description
SRS-29	The System Administrator must use distinct credentials to log in to the dedicated administrative interface.
SRS-30	The Admin Panel shall be secured and inaccessible to standard users.

User Account Management

ID	Requirement Description
SRS-31	The Admin shall have the ability to View and Search for any registered user account.
SRS-32	The Admin shall have the ability to Create, Edit, or Delete user accounts if necessary.
SRS-33	The Admin shall have read-only access to view user-created Categories and processed Transactions for auditing purposes.

2.6 Non-Functional Requirements

2.6.1 Security

- **Account Protection:** User passwords must be stored using a strong one-way hash.
- **Session Security:** The system must use HTTPS for all communication to prevent data interception.
- **User Isolation:** A user's data must be completely isolated and inaccessible to any other user.

2.6.2 Usability

- The interface must be intuitive, clean, and fully mobile-responsive.
- Training time for a normal user to understand the upload and sort process should be minimal (intuitive design).

2.6.3 Reliability

- The system must be available and function correctly 99.5% of the time.
- In any error scenario, the system must provide an appropriate message and retain the state of the user's data (no silent failures).

2.6.4 Performance

- **File Processing Time:** A standard file (up to 500 transactions) must be processed, sorted by the AI, and ready for viewing within 60 seconds.
- The system dashboard should load visualization results in under 3 seconds after processing is complete.

2.6.5 Design Constraints

- **Technology Stack:** Must use Python (Django Framework) for the backend logic.
- **Database:** Designed for use with PostgreSQL for scalability.
- **AI Service:** The system must utilize Gemini API for intelligent transaction classification.
- **Architecture:** The code must follow the MVT (Model-View-Template) architectural pattern.

2.6.6 Data Ownership & Business Rules

- **Data Ownership:** All uploaded data remains the intellectual property of the User.
- **No Third-Party Sharing:** User financial data may not be shared, sold, or exposed to any third party.
- **Persistence:** Processed data remains available until the user explicitly decides to delete it.

2.7 External Interface Requirements

2.7.1 User Interfaces

The user interface will be entirely web-based, optimized for responsive viewing on desktop and mobile devices. Key interfaces include:

- Login/Registration Forms.
- Custom Category Management Form (CRUD interface).
- CSV File Upload Interface with progress/status feedback.

- Interactive BI Dashboard embedded through Metabase.
- A secure Admin Panel (for System Administrator use only).

2.7.2 Hardware Interfaces

No specialized hardware is required. The system will interface with standard internet-supported client devices (PC, laptop, mobile, tablet).

2.7.3 Software Interfaces

The system interfaces with the following software components:

- **Python / Django:** Primary backend web application and API interface.
- **PostgreSQL:** Database management system for data persistence.
- **Python CSV and Django Forms:** Used for data parsing, validation, and persistence into database models.
- **Gemini API:** The core AI service used for transaction classification.
- **Metabase:** Business Intelligence tool used for dashboard visualization and embedded analytics.

2.8 Detailed Functional Requirement Scenarios and Edge Cases

To ensure the system remains stable under varied real-world usage, the core functional requirements must be evaluated against realistic data flows, alternative execution paths, and exception boundaries. This section outlines the sequential operations, alternative states, and failure recovery protocols for the four primary processing hubs of the application.

2.8.1 Sign Up and Authentication Scenarios

The user onboarding process represents the initial security gate of the application. To protect personal financial data, registration and authentication must enforce strict boundaries at the database and application levels.

Sequential Processing Logic

The registration sequence is initiated when a visitor requests the signup form. The Django backend generates a unique Cross-Site Request Forgery (CSRF) token, binds it to the session, and renders the form fields. The user must provide a username, a unique email address, a password, and a password confirmation. Upon submission, the backend performs form-level checks:

1. **Syntax Verification:** The email address is checked against a standard format regex pattern to confirm structural validity.

2. **Duplicate Auditing:** The database is queried to ensure the email address does not match any existing record.

3. **Complexity Evaluation:** The password string is audited against complexity rules, verifying it contains at least eight characters, including numerical and alphabetical values.

4. **Password Matching:** The password and confirmation strings are checked for character-for-character equality.

If all validations pass, the Django authentication module hashes the password string using the PBKDF2 algorithm with a SHA-256 hash. The system opens a database transaction, saves the user record, generates a personalized default category set (e.g., Utilities, Groceries, Transport), commits the transaction, and redirects the user to the login screen.

Alternate Courses and Exception Handling

During registration, input errors or system faults can redirect the execution flow:

- **Duplicate Account Conflict (Alternate Course):** If the database query identifies an existing user account registered under the submitted email, the system halts the creation process. The backend raises a form validation error, re-renders the registration page, and displays a notice stating that the email address is already in use.

- **Password Complexity Failure (Alternate Course):** If the password contains fewer than eight characters or fails complexity rules, the validation script blocks database entry. The system flags the password field and displays a message explaining the length and character requirements.

- **Database Transaction Timeout (Exception Path):** In the event of high database lock contention or server latency, the registration write operation may time out. The application catches the database transaction error, rolls back any partial database updates, logs the system error, and redirects the user to a page explaining that the server is temporarily busy, prompting them to try again.

2.8.2 CSV Ingestion and Parsing Scenarios

The ingestion pipeline converts external financial statement files into internal database records. Since banks and wallet services generate statement CSVs with different layouts, the ingestion service must handle files with structural variations.

Sequential Processing Logic

When an authenticated user requests a file upload, the application validates the file extension and size before opening the data stream. If the file passes the initial boundaries, the Pandas library is initialized to read the CSV content. The sequence executes as follows:

1. **File Stream Buffering:** The uploaded file is read into memory as a byte stream, avoiding disk-write latency.
2. **Header Extraction:** The parser reads the first row to identify the columns and matches them against the expected fields (Date, Description, and Amount) using case-insensitive mapping and predefined aliases.
3. **Data Cleaning:** The system loops through each row in the dataframe. It trims trailing spaces from descriptions and removes currency symbols (such as Rs. or PKR) and thousands-separator commas from the amount values.
4. **Transaction Batching:** The cleaned records are converted into transaction model instances and appended to a list for bulk database insertion.

Once the batch list is fully populated, the Django ORM executes a single bulk insert operation to save the transactions.

Alternate Courses and Exception Handling

Data parsing must account for incomplete files, irregular cell contents, and format mismatches:

- **Missing Column Headers (Alternate Course):** If the header row lacks one of the required columns (Date, Description, or Amount) and no alias matches are found, the Pandas parser halts. The system aborts the upload, discards the buffered stream, and renders a message stating which mandatory column is missing.
- **Non-Numeric Amount Formatting (Alternate Course):** When a row contains non-numeric text in the Amount column (such as "N/A" or "FREE"), the numeric coercion module converts the entry to a null value. The row filter detects this null value and skips the row, recording it in the import log while continuing to process the other rows.
- **Encoding Mismatch Exception (Exception Path):** Financial statements may be encoded in UTF-8 or legacy formats like ISO-8859-1. If the parser encounters characters it cannot read, a decoding exception is raised. The application catches the exception, closes the stream, and displays a message asking the user to save the CSV in standard UTF-8 format.

2.8.3 AI Classification and Inference Scenarios

The Gemini API classification engine automates the assignment of spending categories using semantic natural language processing.

Sequential Processing Logic

When the user triggers the AI sorting function, the backend verifies that the user has custom categories and that there are uncategorized transactions. The classification sequence proceeds as follows:

1. **Taxonomy Loading:** The system fetches the list of custom categories defined by the user.
2. **Transaction Batching:** Uncategorized transactions are divided into batches of 25 to balance API payload size and network response latency.
3. **Prompt Construction:** The engine compiles a structured prompt containing the candidate labels and the batch of transactions.
4. **Gemini Classification:** The text descriptions are evaluated against the user's category list. The Gemini model calculates a confidence score for each category.
5. **Confidence Assessment:** The engine checks the highest confidence score against the 0.35 safety threshold. If it exceeds the threshold, the category is assigned.

The database is then updated with the category assignments.

Alternate Courses and Exception Handling

The AI sorting pipeline must handle low-confidence results and service interruptions:

- **Low-Confidence Bypass (Alternate Course):** If the highest confidence score for a transaction falls below 0.35, the engine flags the prediction as uncertain. The system assigns the default label "Uncategorized" and stores the model's top guess as a suggestion, allowing the user to review the record later.
- **Empty Taxonomy Interception (Alternate Course):** If a user runs the classification engine without defining any categories, the application bypasses model execution. It marks all pending rows as "Uncategorized" and notifies the user to configure custom categories.
- **API Request Timeout (Exception Path):** High network latency or API rate limits may cause classification requests to time out. The application is configured to retry the request once. If the retry fails, the engine halts, rolls back the batch update, and displays a warning to the user, ensuring that no partially sorted records are left in the database.

2.8.4 Dashboard Aggregation and Visualization Scenarios

The analytics module provides secure data visualization dashboards using embedded Metabase resources.

Sequential Processing Logic

When a user accesses the dashboard, the application generates a secure, token-authorized iframe. The sequence runs as follows:

1. **Configuration Audit:** The system checks the environment configuration settings to retrieve the Metabase site URL and embedding secret key.

2. **Token Signature:** The backend compiles a JSON Web Token signed with the HMAC-SHA256 algorithm. The token contains the active user's identification number as a query filter parameter.

3. **URL Construction:** The system appends the signed token to the Metabase dashboard site path, creating a secure embedding URL.

4. **Frontend Embedding:** The Django template receives the URL and renders the Metabase dashboard inside an iframe within a glassmorphic dashboard container. The Metabase server filters the database queries dynamically using the user ID parameter.

2.8.5 Alternate Courses and Exception Handling

The dashboard aggregation system must handle empty datasets and authorization issues:

- **Missing Key Interception (Alternate Course):** If the Metabase secret key is missing from environment configurations, the dashboard view intercepts the error. It blocks the iframe generation and renders a warning banner explaining how to configure the environment settings.
- **Expired Session Token (Alternate Course):** The JSON Web Token contains an expiration timestamp set to 10 minutes. If the user keeps the dashboard open past this limit, Metabase displays a signature expired error. Refreshing the dashboard page generates a fresh token, restoring the view.
- **Zero Record State (Exception Path):** If the user has no transactions, the database queries return empty sets. Metabase displays empty visual states on the dashboard, explaining that data needs to be uploaded to populate the charts.

Chapter 3

System Design and Use Case Analysis

3.1 Use Case Diagram

The Use Case Diagram provides a high-level overview of the system's interactions with the primary actors (User and System Administrator).

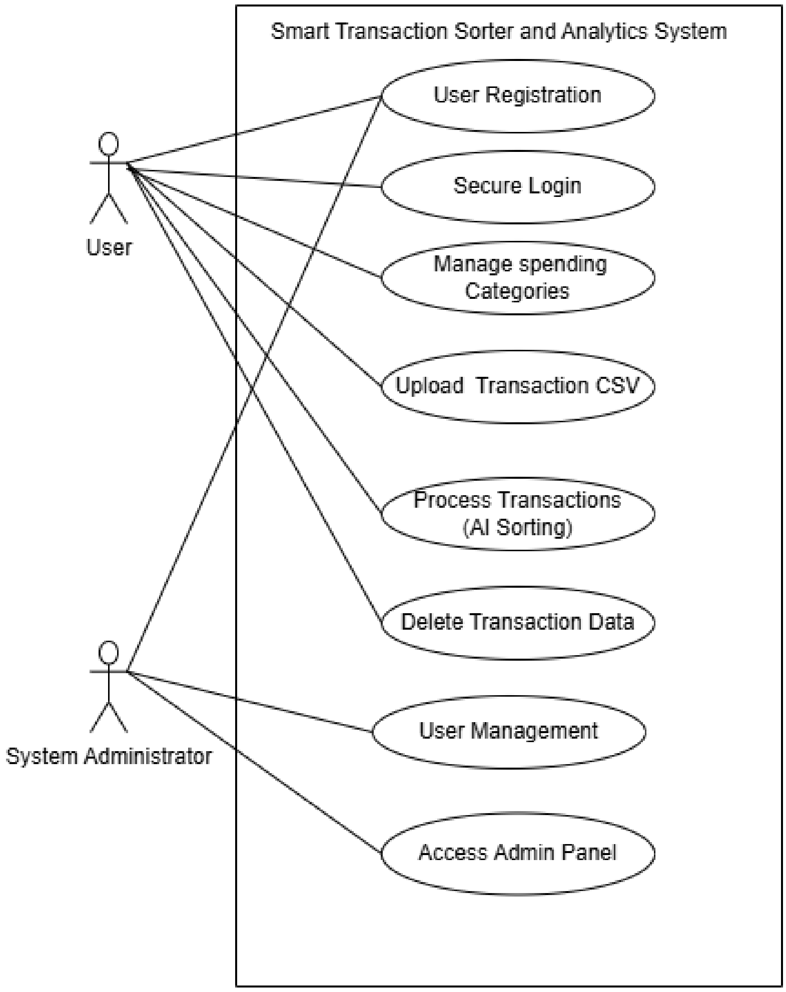

Figure 3.1: Smart Transaction Sorter System Use Case Diagram

3.2 Fully Dressed Use Case Scenarios

3.2.1 User Access & Authentication

Use Case 1.1: User Registration

Use Case ID	1.1
Use Case Name	User Registration
Actor	User
Description	A new user creates a secure account to access the system features.
Preconditions	User has a valid email address and internet access.
Postconditions	A new user account is created in the database, and the user is redirected to the login page.
Priority	High
Normal Course	1. User navigates to the application landing page. 2. User selects ”Register”. 3. System displays the registration form. 4. User enters Name, Email, and Password (twice). 5. System validates the input format. 6. System hashes the password and saves the new user record. 7. System displays a success message.
Alternative Courses	**5a. Weak Password:** i. System detects password does not meet security complexity requirements. ii. System prompts user to choose a stronger password. **6a. Email Already Exists:** i. Database check confirms the email is already registered. ii. System displays error: ”Account already exists”. iii. System prompts user for Login.
Exceptions	Database connection failure prevents account creation.
Special Req.	Passwords must be hashed using a strong algorithm (e.g., PBKDF2 or Argon2) before storage.

Use Case 1.2: Secure Login

Use Case ID	1.2
Use Case Name	Secure Login
Actor	User, System Administrator
Description	Users or Admins log in to access their respective dashboards.
Preconditions	User/Admin must have a registered account.
Postconditions	Actor is authenticated, a session is established, and they are redirected to their authorized dashboard.
Priority	High
Normal Course	1. Actor navigates to the Login page. 2. Actor enters Email and Password. 3. System verifies credentials against the database. 4. System generates a secure session token. 5. System identifies the role (User or Admin) and redirects to the appropriate Dashboard.
Alternative Courses	**3a. Invalid Credentials:** i. System detects email/password do not match. ii. System displays ”Invalid Login” error. iii. System allows retry (limit 5 attempts).
Exceptions	Account is locked due to too many failed attempts.
Special Req.	Communication must occur over HTTPS.

3.2.2 Core Configuration

Use Case 2.1: Manage Spending Categories

Use Case ID	2.1
Use Case Name	Manage Spending Categories
Actor	User
Description	User defines the custom labels (e.g., ”Gym”, ”Groceries”) that the AI will use for sorting.
Preconditions	User is logged in.
Postconditions	The list of categories is updated in the database and ready for the AI engine.
Priority	High
Normal Course	1. User navigates to ”Manage Categories”. 2. System displays current list of categories. 3. User clicks ”Add New Category”. 4. User types a label name. 5. System saves the category. 6. System confirms update.
Alternative Courses	**2a. Delete Category:** i. User selects ”Delete” on an existing category. ii. System removes the category from the list. **5a. Duplicate Category:** i. User tries to add a category that already exists. ii. System prevents save and shows a duplicate error.
Special Req.	Categories must be unique per user.

3.2.3 Data Processing (The AI Core)

Use Case 3.1: Upload Transaction CSV

Use Case ID	3.1
Use Case Name	Upload Transaction CSV
Actor	User
Description	User uploads a raw CSV file from their bank/wallet for processing.
Preconditions	User is logged in and has a valid CSV file.
Postconditions	File is parsed, and all transactions are saved to the database with an ’Uncategorized’ status.
Priority	High
Normal Course	1. User navigates to ”Upload” section. 2. User selects a file from local device. 3. System scans file type. 4. System validates required columns (Date, Description, Amount). 5. System parses the file and saves all transactions to the database. 6. System confirms successful upload.
Alternative Courses	**3a. Invalid File Format:** i. System detects file extension is not .csv. ii. System rejects the file. **4a. Missing Columns:** i. System validation fails (missing ”Amount” column). ii. System rejects file. iii. System tells User exactly which column is missing.
Exceptions	File size exceeds server limit (e.g., >10MB).
Special Req.	Validation must happen immediately before any processing.

Use Case 3.2: Process Transactions (AI Sorting)

Use Case ID	3.2
Use Case Name	Process Transactions (AI Sorting)
Actor	System (Triggered by User)
Description	The AI engine analyzes transaction descriptions and assigns them to User's custom categories.
Preconditions	Valid CSV uploaded (UC-3.1) and Categories exist (UC-2.1).
Postconditions	All transactions are categorized and saved to the database.
Priority	High
Normal Course	1. User initiates "Process". 2. System retrieves User's Category List. 3. System feeds text descriptions into the AI Classification Engine. 4. AI determines best category match for each row. 5. System saves classified data to database. 6. System notifies User of completion.
Alternative Courses	**4a. Low Confidence:** i. AI model confidence score is below threshold. ii. System assigns a default "Uncategorized" label. iii. System flags transaction for manual review.
Exceptions	AI Service timeout.
Special Req.	Processing 500 records must take < 60 seconds.

3.2.4 Analytics & Maintenance

Use Case 4.1: View Analytics Dashboard

Use Case ID	4.1
Use Case Name	View Analytics Dashboard
Actor	User
Description	User views interactive charts summarizing their financial data.
Preconditions	Data has been processed (UC-3.2).
Postconditions	User gains insight into spending habits.
Priority	High
Normal Course	1. User clicks ”Dashboard”. 2. System aggregates total spending per category. 3. System renders charts (Pie/Bar) using plotting libraries. 4. User filters view by Date Range (optional).
Alternative Courses	**2a. No Data:** i. Database query returns no records. ii. System displays ”Upload data to see insights” prompt.
Special Req.	Dashboard must be mobile responsive.

Use Case 4.2: Delete Transaction Data

Use Case ID	4.2
Use Case Name	Delete Transaction Data
Actor	User
Description	User permanently removes uploaded data from the system.
Preconditions	User has existing data.
Postconditions	Data is permanently erased from database.
Priority	Medium
Normal Course	1. User navigates to ”Data Management”. 2. User selects a file batch or ”Delete All”. 3. System displays ”Are you sure?” confirmation warning. 4. User confirms. 5. System performs hard delete of records. 6. Dashboard updates to show empty state.
Alternative Courses	**4a. Cancel Delete:** i. User clicks ”Cancel” at confirmation. ii. System retains data and cancels the operation.
Special Req.	Data must be unrecoverable after deletion (Privacy Rule).

3.2.5 Administration

Use Case 5.1: Admin User Management

Use Case ID	5.1
Use Case Name	Admin User Management
Actor	System Administrator
Description	Admin manages user accounts and views system usage stats.
Preconditions	Admin is logged in and authorized.
Postconditions	User account status is updated.
Priority	Low
Normal Course	1. Admin navigates to the Administrative Panel. 2. Admin searches for a User by email. 3. System displays User details. 4. Admin selects ”Edit” or ”Delete”. 5. System processes the administrative action.
Alternative Courses	**3a. Audit View:** i. Admin selects ”View Details”. ii. System displays read-only list of User’s categories for support purposes.
Exceptions	Admin cannot view User’s raw passwords (hashed).
Special Req.	Admin actions must be logged.

3.3 Process Modeling and System Analysis

The use cases above describe the system from the perspective of its actors. Process modeling describes the same system from the perspective of runtime behavior: what happens first, which component responds, and how data moves from raw input toward categorized analytics. For this project, process modeling is especially important because the application is a multi-step workflow that begins with authentication, depends on user-defined categories, accepts transaction files, invokes an AI classification service, persists the result, and finally visualizes categorized data through a BI layer.

3.3.1 System Activity Diagram

The Activity Diagram illustrates the operational workflow of the system, detailing the flow of control between the User, the System, and the Intelligent Sorting Module. It captures the sequential steps from authentication to the final visualization of analytics. The diagram is intentionally broad because it represents the user journey rather than implementation classes.

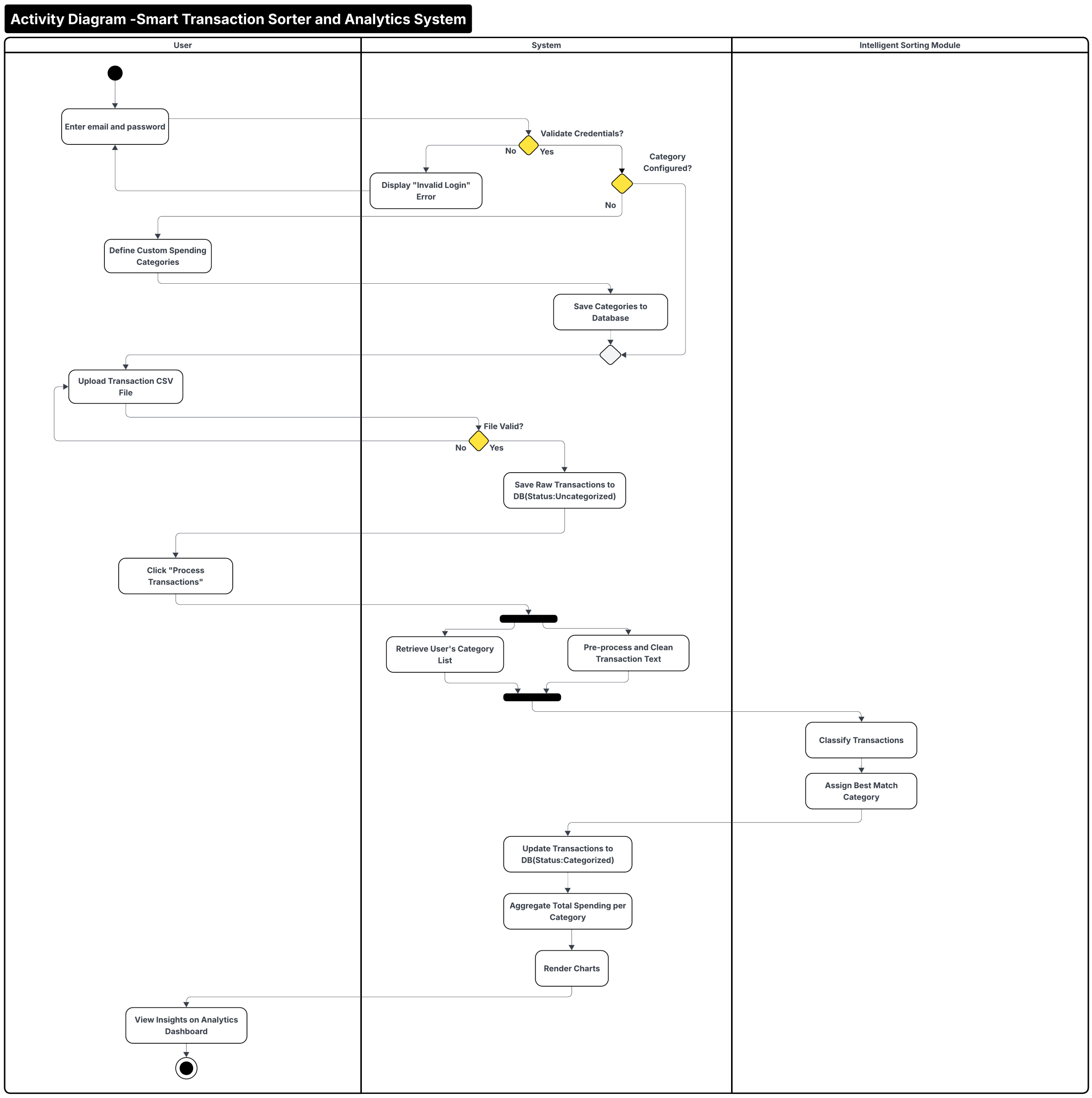

Figure 3.2: Activity Diagram: Smart Transaction Sorter and Analytics System

The first decision point in the activity diagram is credential validation. This corresponds to Django authentication in the implemented application. If authentication fails, the user cannot reach protected screens such as category management, CSV upload, AI sorting, or analytics. After authentication, the workflow checks whether the user has configured spending categories. This reflects a core design rule: the AI classifier must choose from a valid user-specific category list rather than from a fixed global taxonomy.

The second major decision point is file validation. The system must reject files that are structurally invalid before saving rows to the database. This prevents downstream classification from operating on malformed records and allows the user to correct the upload problem immediately. Once rows are saved, the system treats uncategorized transactions as stable input for AI processing. This separation between upload and AI sorting is deliberate because it makes the workflow recoverable. If the AI service is unavailable or returns an error, the uploaded transaction rows remain stored and can be processed later.

3.3.2 System Sequence Diagram

The System Sequence Diagram (SSD) depicts the interactions between the User, the Smart Transaction System, and the AI Engine over time. It details the specific messages exchanged during the core processes of authentication, data processing, and analytics.

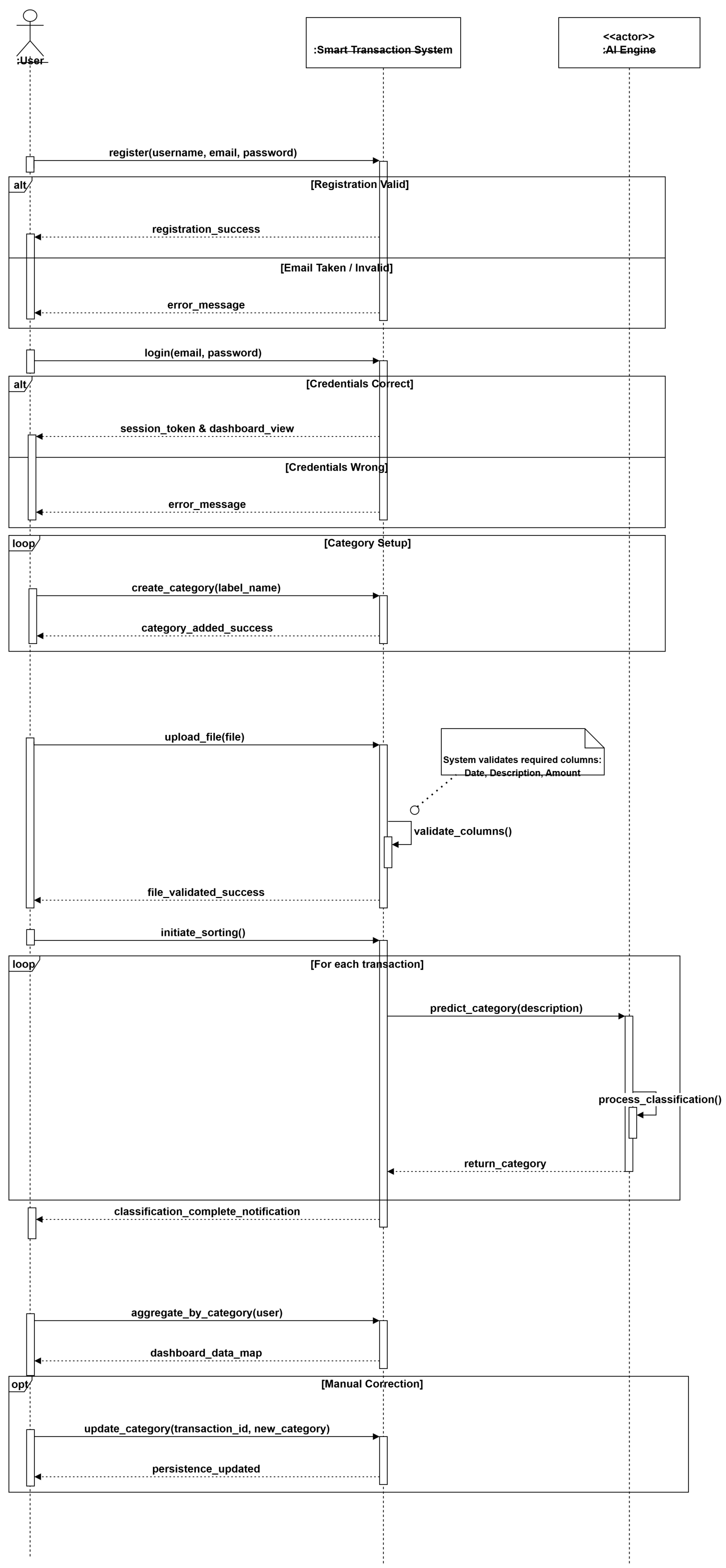

Figure 3.3: System Sequence Diagram (SSD) for Core Workflow

The SSD emphasizes the external contract of the system. The user does not call the database or AI service directly. Instead, the web application mediates each interaction: registration, login, category setup, CSV upload, AI processing, and dashboard generation. This mediation is central to the system's security posture because it allows the application layer to check authentication, filter database queries by the logged-in user, validate uploaded files, and validate AI output before any persistent state is changed.

3.4 Database Design

The database design serves as the foundational data layer for the Smart Transaction Sorter. It ensures that user profiles, custom categories, default category suggestions, and transaction records are stored efficiently and securely. The confirmed final deployment target is PostgreSQL, while the relational model is expressed through Django models. PostgreSQL is appropriate for the final version because it provides durable relational storage, indexing, referential integrity, and scalability for analytical workloads connected to Metabase.

3.4.1 Entity Relationship Diagram

The Entity Relationship Diagram (Figure 3.4) visualizes the logical structure of the database. It defines the entities such as Users, Categories, and Transactions and the relationships between them. Key relationships include the one-to-many association between a User and their Categories, the one-to-many association between a User and Transactions, and the optional many-to-one relationship from Transaction to Category.

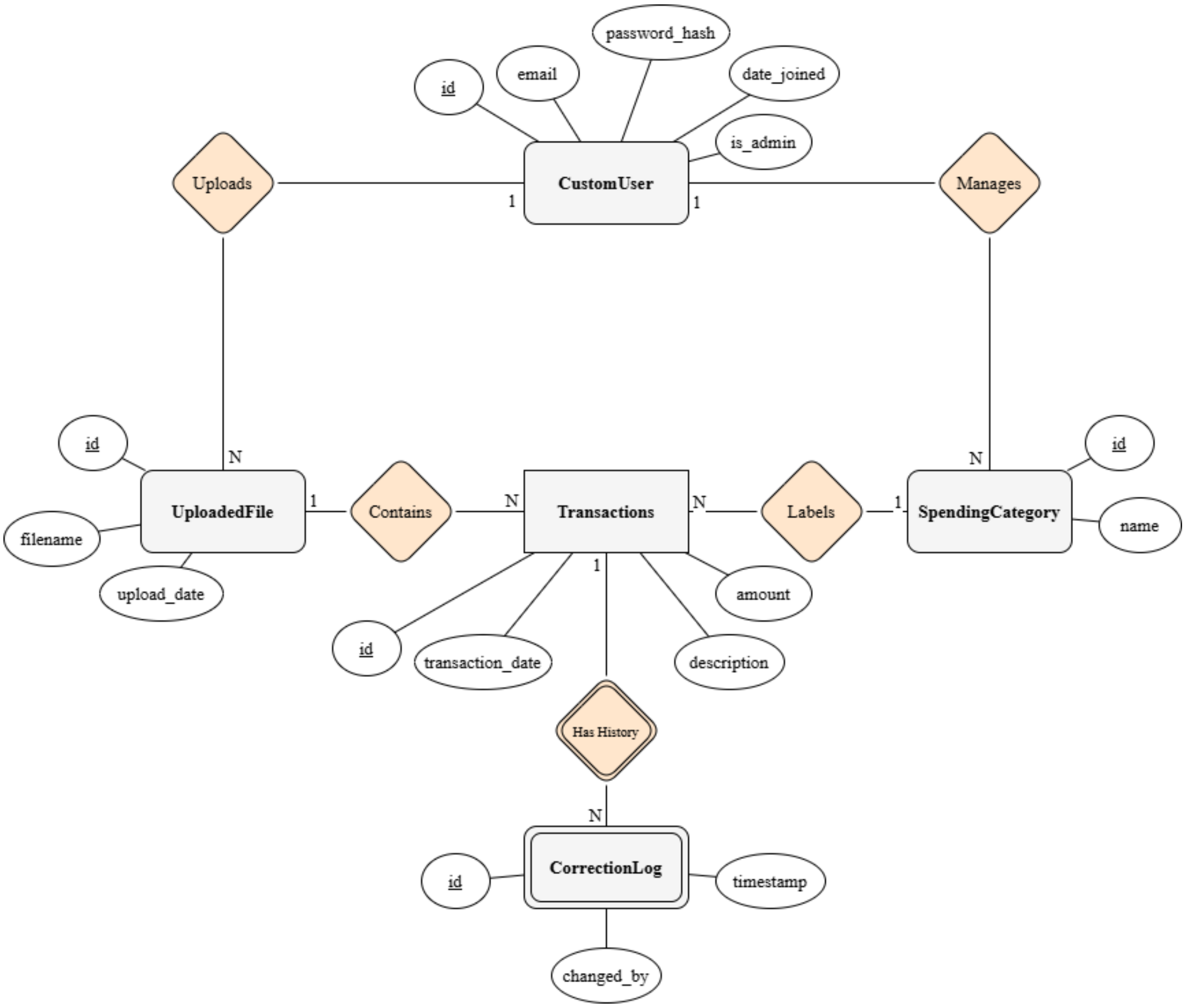

Figure 3.4: Entity Relationship Diagram (ERD)

The ERD reflects the principle of user isolation. A category is not merely a label such as Groceries or Rent; it belongs to a specific user. This allows two users to create categories with the same visible name without sharing financial data or classification behavior. A transaction also belongs directly to a user. This is necessary because a transaction can temporarily have no category before AI processing, and it may return to an uncategorized state when confidence is low or a category is removed. Direct ownership prevents uncategorized rows from becoming orphaned.

3.5 Application Architecture

The application architecture follows a modular Django Model-View-Template design. Django models represent persistent data, views coordinate request handling and business workflows, templates render the user interface, forms validate user input, and URL configurations map browser requests to the correct application behavior. This separation of concerns improves maintainability because authentication, category management, CSV ingestion, AI sorting, and analytics embedding can be reasoned about as distinct modules while still sharing the same user session and database.

3.5.1 Class Diagram

The Class Diagram (Figure 3.5) depicts the static structure of the system. It illustrates the system's classes, their attributes, methods, and relationships. The diagram is retained from the original design documentation because it captures the intended relationships among users, transaction data, category labels, AI sorting logic, and analytics behavior.

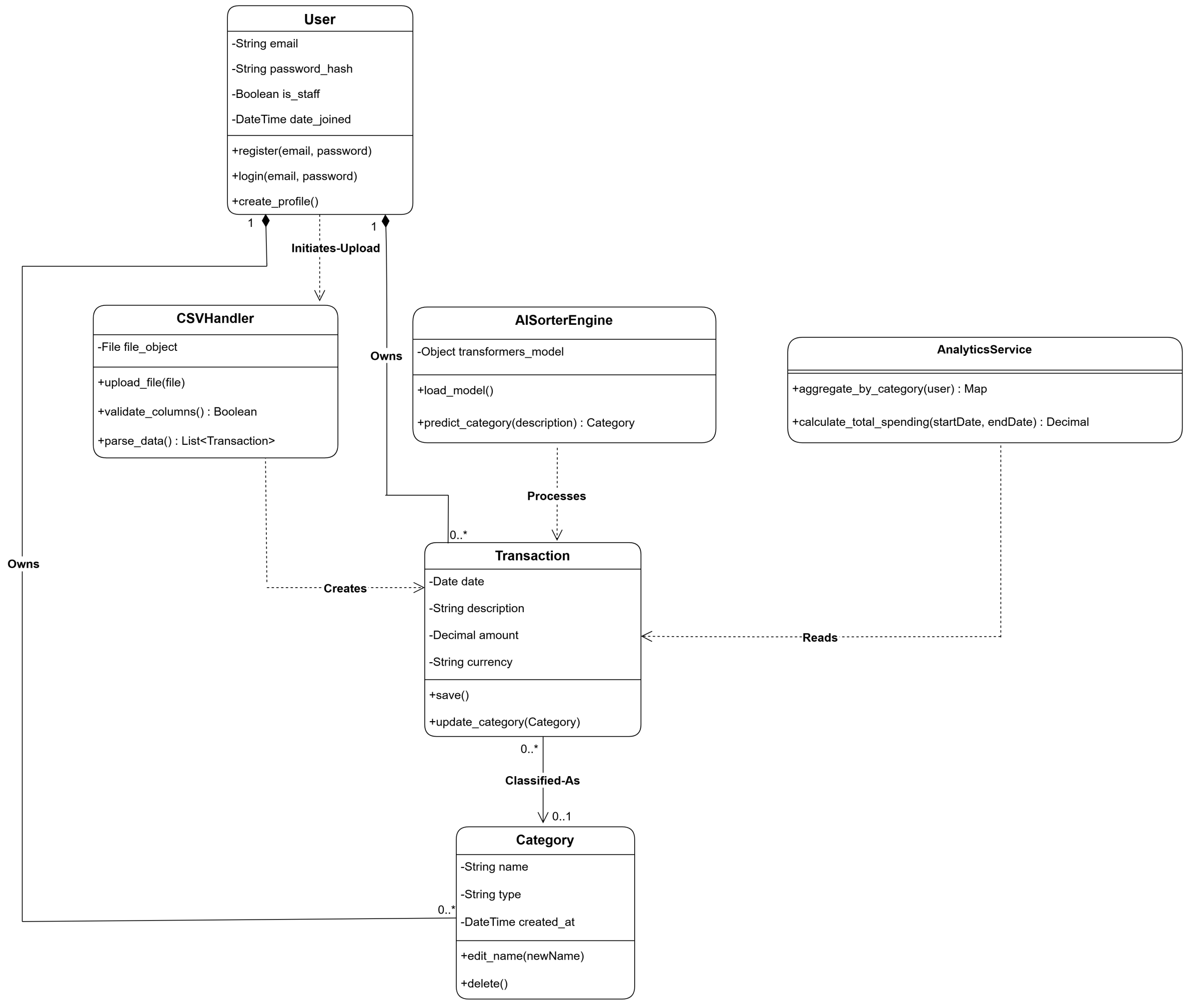

Figure 3.5: System Class Diagram

In implementation, the class-level design is realized through Django models and service classes. User identity is provided by Django's authentication model. Category and Transaction are implemented as Django models in the pages application. CSV handling is implemented in the transaction upload view and form. AI sorting is implemented as a dedicated service module containing batching, classification, response validation, and orchestration responsibilities. Analytics is implemented through a Django view that generates a secure Metabase dashboard URL.

3.5.2 System Data Flow and Component Processing Lifecycle

To fully understand the runtime execution of the application, it is necessary to trace the lifecycle of transactional data as it traverses different modules, format boundaries, and processing states. The system's processing lifecycle is divided into four distinct phases: ingestion, semantic classification, persistence, and analytics visualization.

Data Ingestion and Schema Normalization

The ingestion phase begins when the client browser transmits a file upload request containing a raw transaction CSV file. This request is captured by the application's URL routing module and dispatched to the upload handler. Before reading the file stream, the handler performs form-level validations, checking that the file size is within safety limits and that the file extension is strictly .csv to prevent formatting issues. The system provides specific errors for common non-csv extensions like .xlsx, .pdf, or .json, suggesting the correct format to the user. Once validated, the byte stream is decoded using a UTF-8 character representation and passed to the built-in CSV dictionary reader.

The parser reads the first row to identify the column headers. It maps the headers to the application's database fields using an internal alias mapping dictionary. This is necessary because different financial institutions use different naming conventions, such as Date or Posting Date, and Description or Memo. Once the columns are aligned, the system initializes an iterative parsing loop. Each row is checked for completeness; if cell values in the Date, Description, or Amount columns are missing, the row is skipped and logged as an error.

At the same time, the Date values are converted using a sequence of date parser templates to ensure they conform to the database-compatible YYYY-MM-DD format. The values in the Amount column are normalized by removing currency markers (such as PKR, Rs., or commas) and casting the cleaned strings to fixed-point decimal values.

Semantic Classification and Machine Learning Inference

After the ingestion pipeline parses and normalizes the transactions, the user can run the AI sorting module. The classification service collects the uncategorized transaction records and retrieves the custom category list defined by the user. Uncategorized transactions are divided into batches of 25 to balance API payload size and network response latency.

For each batch, the engine compiles a structured prompt containing the candidate labels and the batch of transactions. This structured prompt is passed to the Gemini API using the gemini-2.0-flash model. The model does not use exact keyword matching. Instead, it evaluates the semantic meaning of transaction text descriptions against the valid category taxonomy and returns a strictly formatted JSON array.

The model computes a probability score for each category. If the highest score meets the 0.35 confidence threshold, the category is selected. If the confidence score falls below this safety line, or if the model suggests a category that is not in the user's defined list, the engine assigns the default label

"Uncategorized". It preserves the model's top guess as a suggested category in the database for manual confirmation.

Database Relational Persistence

Once the classification engine completes processing for a batch of transactions, it passes the data objects to the database access layer. The system maps the assigned category names to primary key identifiers in the database.

To avoid database locking delays, the application collects the categorized transaction records and performs a bulk update using the ORM. This bulk update modifies the category foreign key, the AI confidence score, the classification status flag, and the suggested category fields in a single database query.

Relational integrity is protected through foreign key constraints: the category foreign key is configured with a null-on-delete constraint. This ensures that if a user deletes a custom category, the associated transactions revert to an uncategorized state rather than being deleted from the database.

Analytical Aggregation and Metabase Visualization

The final phase of the processing lifecycle is analytics presentation. When the user requests the dashboard view, the backend query engine aggregates the spending data.

The system does not compile local charts using browser scripting. Instead, it requests a secure dashboard embedding from an external Metabase business intelligence server. The Django backend generates a JSON Web Token signed with a secret key using the HMAC-SHA256 algorithm. The token contains the active user's identification number as a parameter to ensure that the Metabase database queries are filtered specifically for that user.

The client browser receives this secure token in an iframe source URL and requests the rendered dashboard. The Metabase server processes the dashboard layouts and delivers responsive, interactive charts directly into the user interface, ensuring complete data separation between different user profiles.

3.5.3 Interaction Design

The following sequence diagrams describe the most important workflows in the system. They remain unchanged because they still represent the core workflow accurately: authentication, category setup, CSV upload, AI processing, dashboard access, data deletion as a privacy requirement, and administration. The construction chapter later explains where each workflow is implemented in the codebase.

User Registration Workflow

This interaction details the process of creating a new user account, including validation of unique identity fields and secure password hashing before storage.

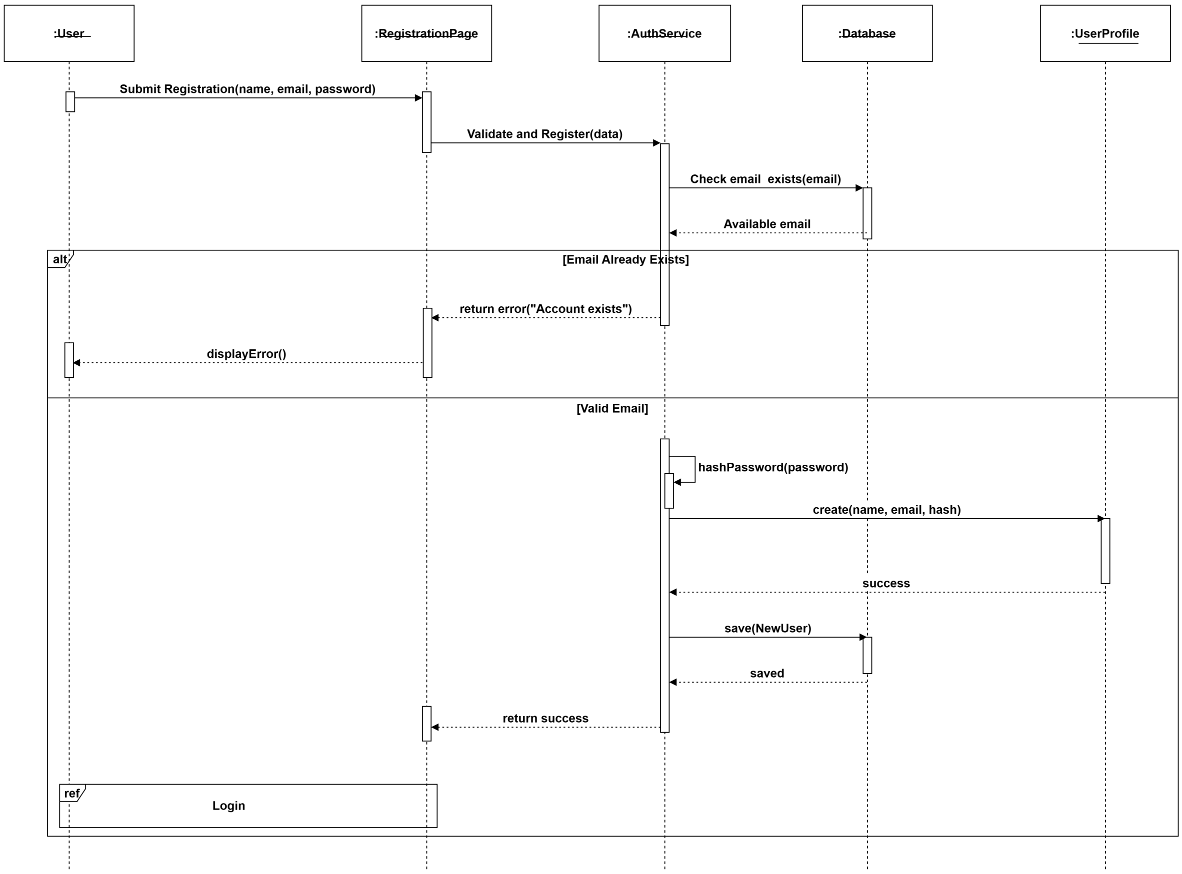

Figure 3.6: Sequence Diagram: User Registration

Secure Login Workflow

This diagram illustrates the authentication process, verifying credentials against the database and establishing a protected user session upon success.

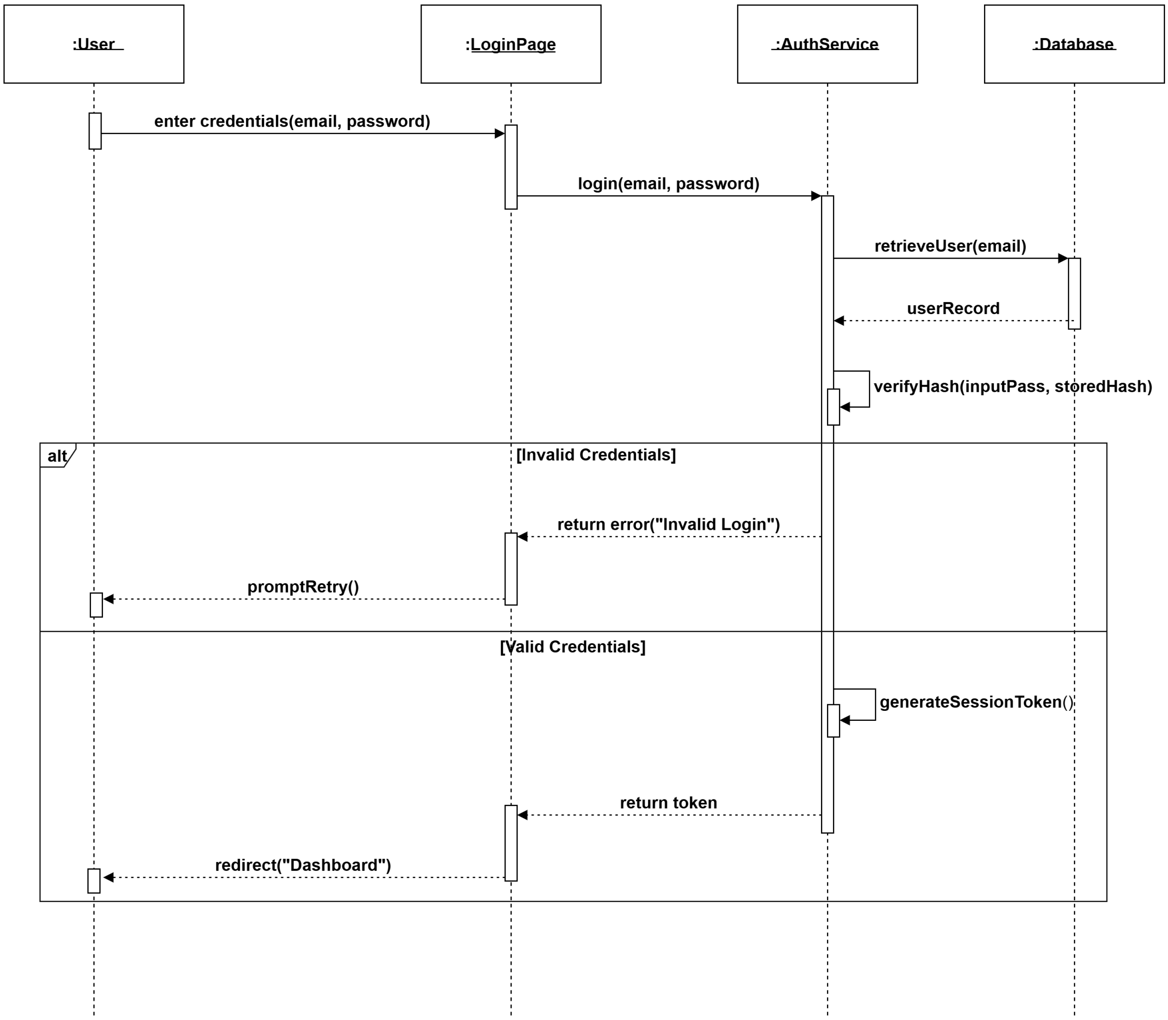

Figure 3.7: Sequence Diagram: User Login

Manage Spending Categories Workflow

This flow demonstrates how a user defines custom spending labels and how the system prevents duplicate category entries for the same user.

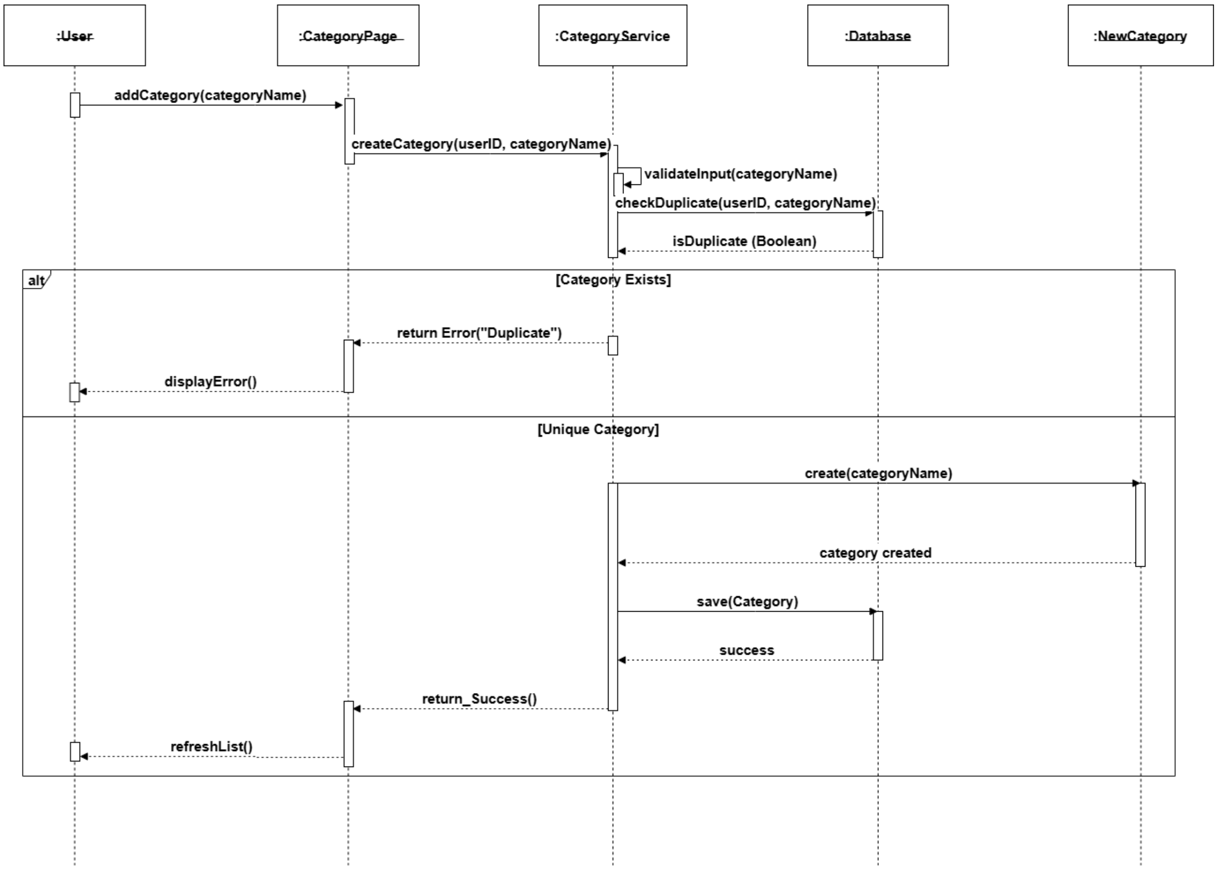

Figure 3.8: Sequence Diagram: Manage Custom Categories

CSV Data Upload Workflow

This diagram covers the data ingestion process, including file extension validation, structure verification, and saving raw transaction data.

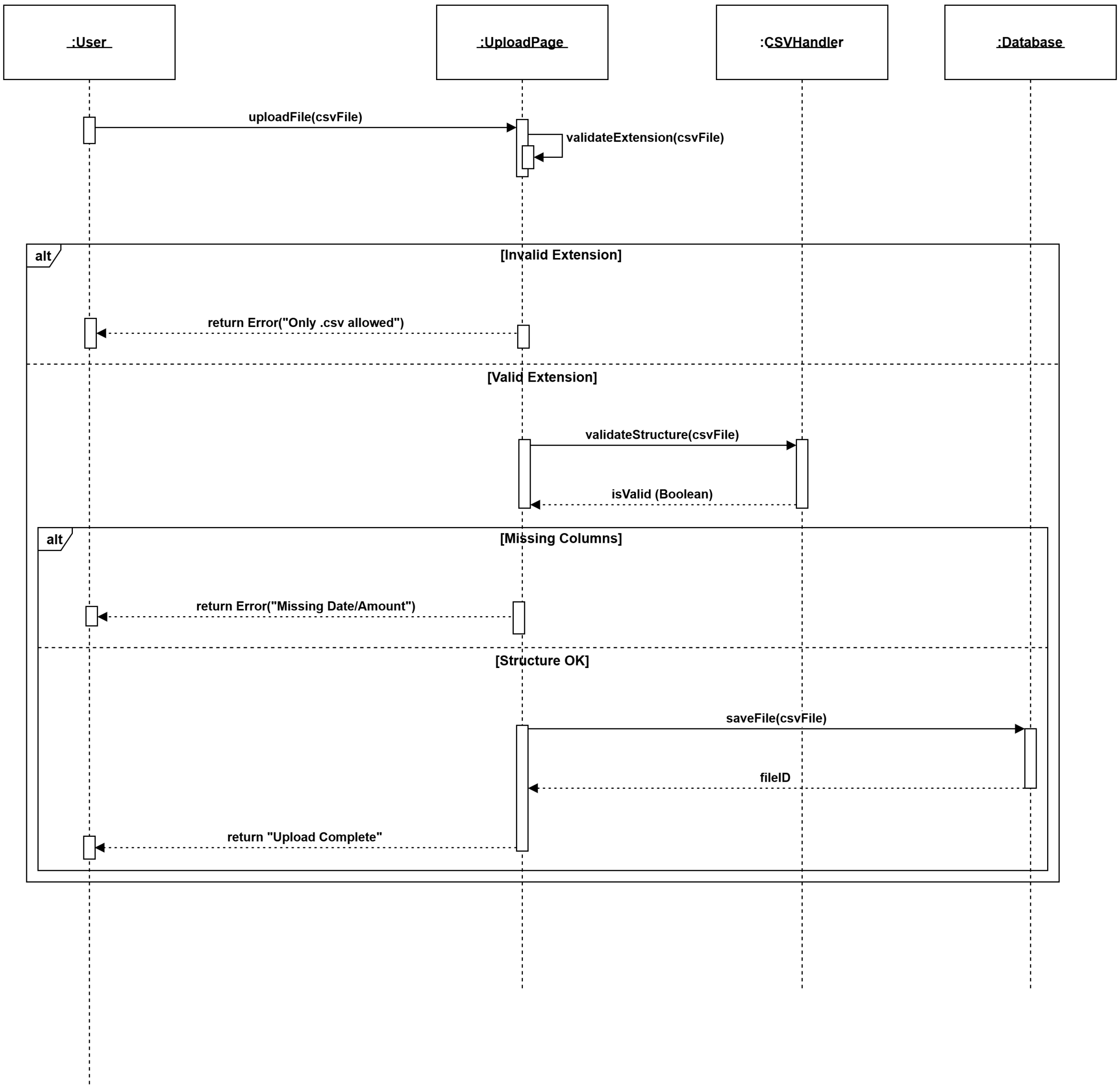

Figure 3.9: Sequence Diagram: CSV Data Upload

AI Transaction Processing Workflow

This is the core system process. It depicts the loop where the AI Engine reads each transaction, predicts the best category based on the user's list, and updates the record.

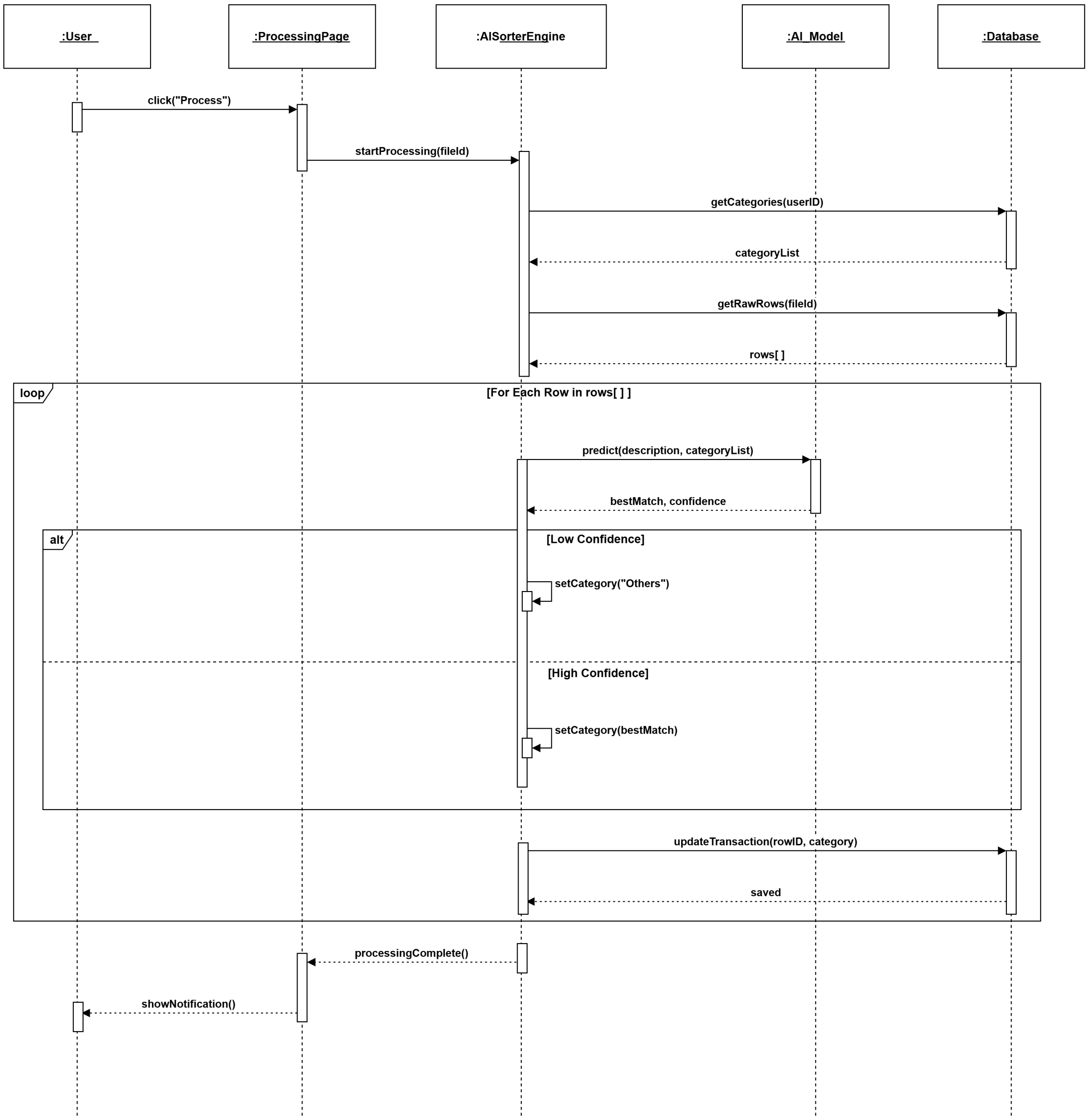

Figure 3.10: Sequence Diagram: AI Transaction Processing

Dashboard Analytics Workflow

This interaction shows how the system exposes categorized data to the BI layer and delivers analytics to the authenticated user through the dashboard interface.

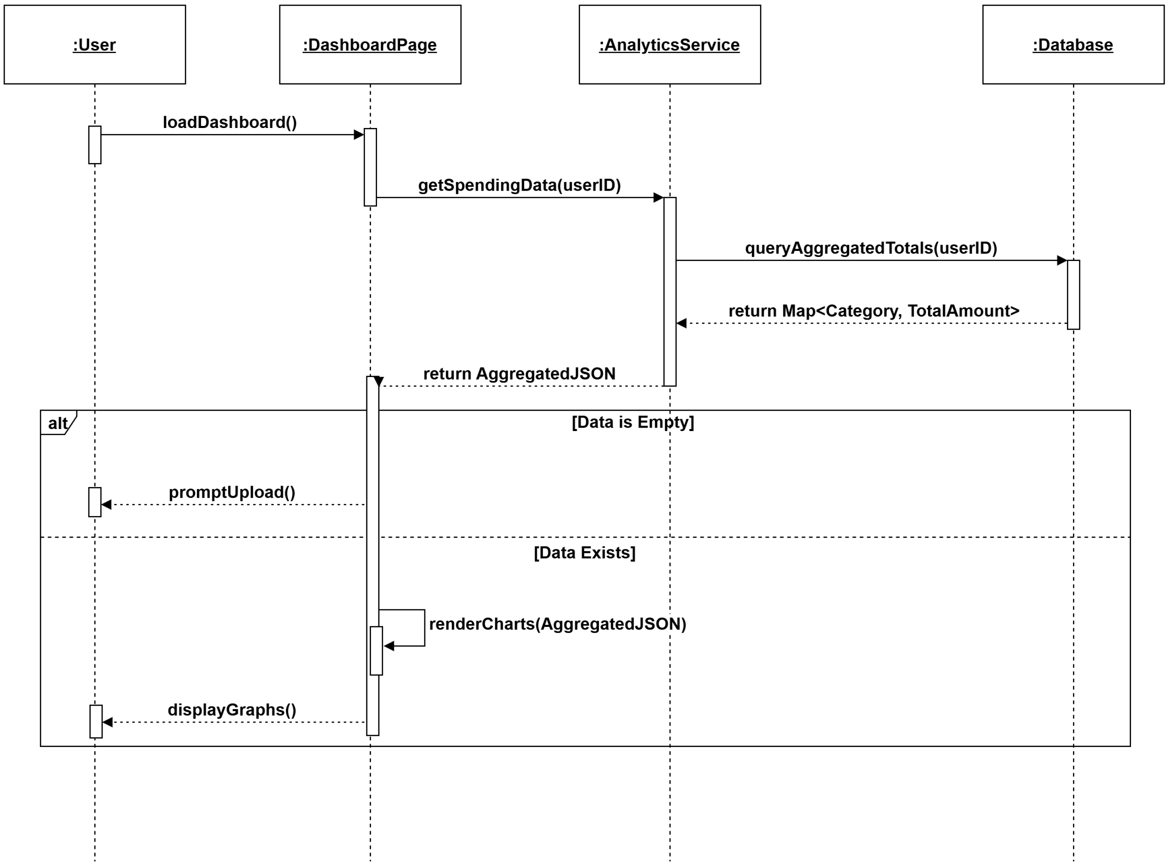

Figure 3.11: Sequence Diagram: Dashboard Analytics

Data Deletion Workflow

This diagram illustrates the privacy feature allowing users to permanently erase their data, including confirmation logic to reduce accidental deletion.

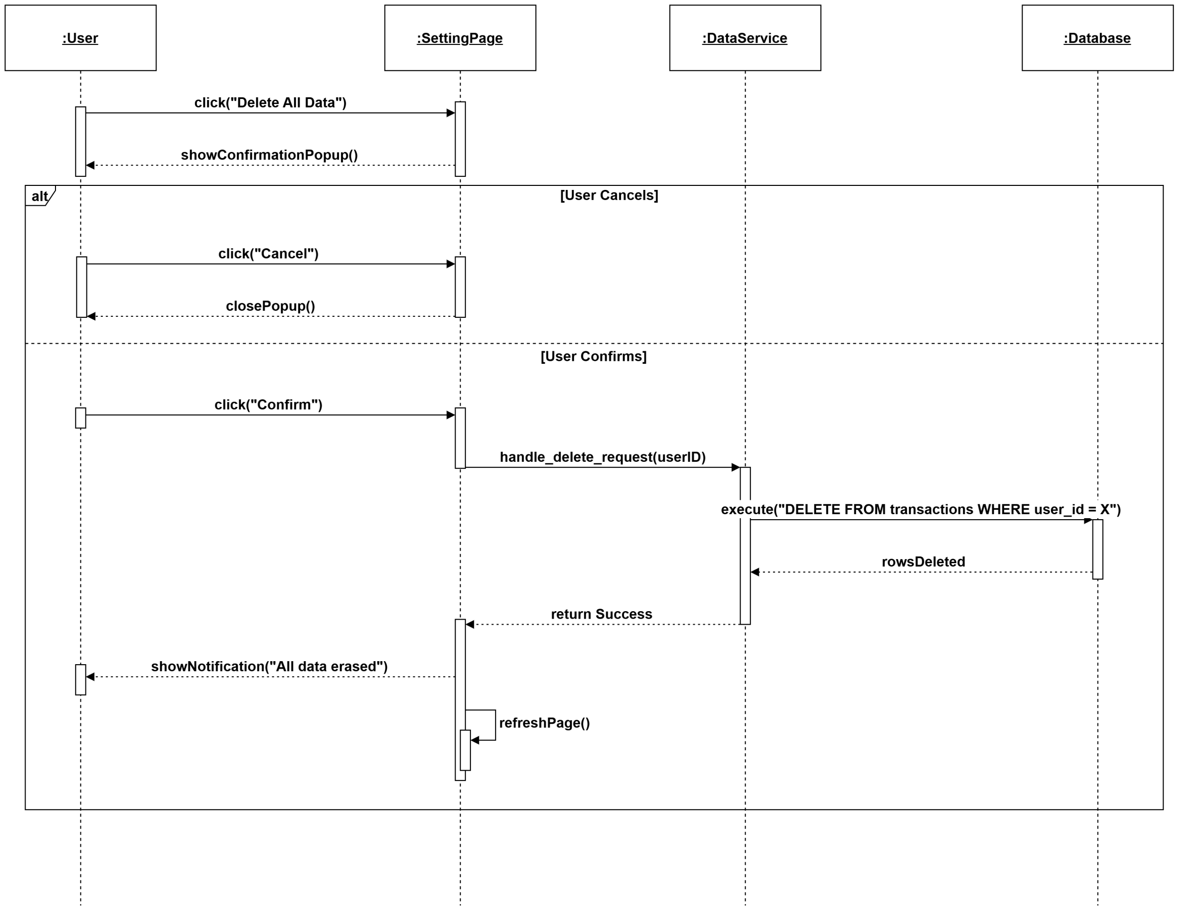

Figure 3.12: Sequence Diagram: Delete All User Data

Admin User Management Workflow

This final diagram details the administrative capabilities, such as searching for specific users in the system and performing account management actions.

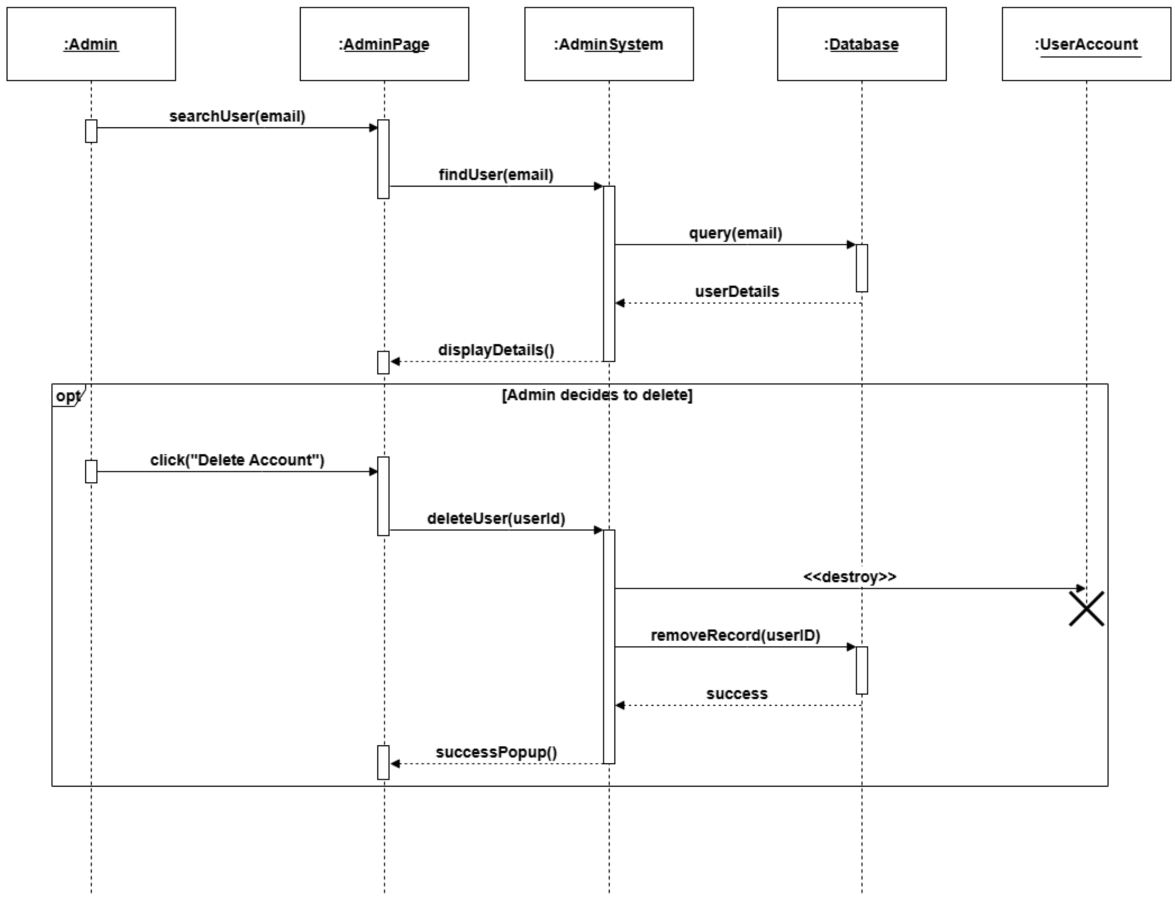

Figure 3.13: Sequence Diagram: Admin User Management

3.6 Physical Architecture and Deployment View

The Physical Architecture describes the hardware and software environment in which the system runs. It outlines the deployment nodes, including the client device, the Django application server, the PostgreSQL database server, the Gemini API service, and the Metabase BI layer. In the final target architecture, the browser communicates with the Django application over HTTPS, Django reads and writes application data to PostgreSQL, Gemini API is called for classification, and Metabase reads analytical data for dashboard generation.

- **Client Node:** Represents the end-user's device running a modern web browser to access the application.
- **Application Server Node:** Hosts the Django web application, URL routing, views, templates, forms, and AI orchestration logic.
- **Database Node:** Hosts PostgreSQL for persistent relational storage of users, categories, default categories, and transactions.
- **AI Service Node:** Represents Gemini API, which receives structured prompts and returns JSON classifications.

- **BI Node:** Represents Metabase, which provides the dashboard embedded inside the Django interface.

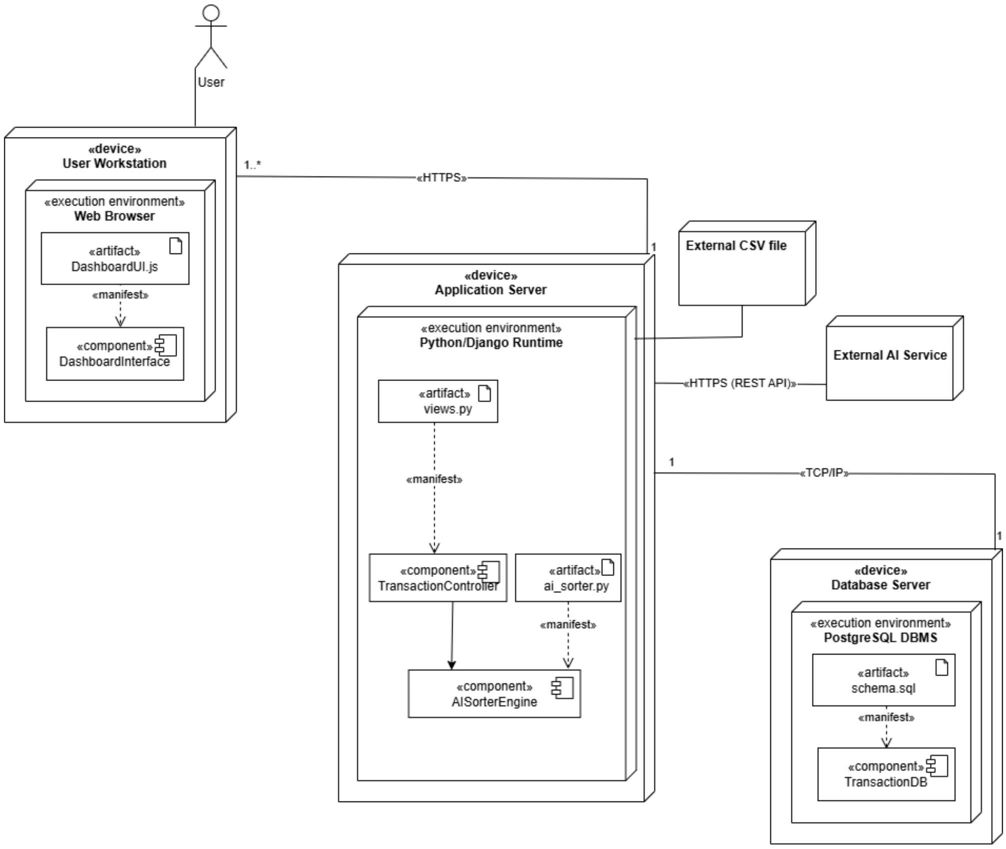

Figure 3.14: System Deployment Architecture

This deployment design separates operational responsibilities. Django owns application security and user workflow. PostgreSQL owns durable relational data. Gemini API owns language-model based classification. Metabase owns dashboard presentation. Separating these responsibilities keeps the application maintainable because the classification service and dashboard service can evolve without rewriting the core transaction upload and persistence logic.

Chapter 4

Construction

4.1 Local Development Hardware Ecosystem and Environmental Baseline

4.1.1 Hardware Performance and Resource Demands

The implementation and local testing of the web application were carried out on a computer equipped with an 11th Gen Intel Core i5 processor running at 2.6 GHz. This multi-core setup was selected to handle concurrent request-response handling of the Django development server while simultaneously running model inference. Deep learning model inference is computationally expensive, meaning that standard processing capacities must be assessed to avoid bottlenecks or latency during batch execution.

The multi-core architecture of the processor enabled the system to execute background threads without causing the user interface to freeze. During batch processing, Django's server thread handles the HTTP request-response cycle while the machine learning classifier processes transactions. This division of labor prevented request timeouts on the client side during high-density classification workloads.

Standard processing baselines were evaluated by running multiple simultaneous client requests. The processor maintained a stable core temperature and clock speed, confirming that consumer-grade processors can support the classification of hundreds of records locally, provided that the system memory is sized appropriately.

4.1.2 Memory Allocation and Storage Speeds

To ensure system stability during the execution of the application, the system was configured with 16 GB of DDR4 RAM. Since the machine learning model runs in the cloud via the Gemini API, local memory demands are very light. However, having 16 GB of memory ensures that standard operations, database cache pools, and other Django worker tasks remain active in RAM without virtual disk swapping. Additionally, a 256 GB solid-state drive (SSD) was used to facilitate rapid read operations when writing transaction datasets and loading page templates.

The cloud-hosted classification architecture eliminates the need to hold massive model parameter matrices in local memory. This is a significant improvement over running large language models locally, which would require gigabytes of dedicated RAM and specialized hardware processors. With a remote API client, the local web server remains responsive and can be deployed on standard, low-cost virtual private servers.

The storage read speeds of the 256 GB SSD further reduced startup latency. Loading the web application components and initializing database connections takes less than 4 seconds, compared to over 20 seconds on standard mechanical hard drives. This setup ensures that the system is ready to receive requests immediately after boot.

4.2 Core Software Environment and Clean Framework Selection

4.2.1 Runtime Configurations and IDE Setup

The software stack consists of open-source Python tools chosen for stability in web development and data processing workflows. The host system ran Windows 11 Home Edition, configured with Python version 3.10.11. Code development and debugging were conducted in Visual Studio Code version 1.85, using Pylance and Django development extensions for structural validation.

Operating system environment variables were used to manage configuration parameters, such as the Gemini API credentials and Metabase embedding parameters. Storing configuration parameters outside the workspace prevented sensitive keys from being exposed in the repository and ensured that the codebase remained lightweight and portable.

The IDE environment was configured with automatic formatting and syntax validation tools. Visual Studio Code was set up to perform PEP 8 compliance checks automatically, reducing code quality errors during prototyping. Pylance was configured to perform static type checking, which helped catch type mismatch errors before runtime execution.

4.2.2 Architectural Pattern and Local Prototyping

The backend framework is Django version 4.2.7, which implements the Model-View-Template (MVT) design pattern. This architecture establishes a clear separation of data schemas, request handling logic, and user-facing layouts. For initial local development and database schema testing, SQLite 3 was utilized as the storage layer because it simplifies initial prototyping while maintaining logical compatibility with a future transition to PostgreSQL.

The Model layer defines the database schema, including validation rules and relationships. The View layer handles the request-response lifecycle, coordinating authentication, CSV parsing, and AI classification. The Template layer renders the HTML layouts, incorporating JavaScript for dynamic updates.

SQLite 3 was configured with write-ahead logging (WAL) enabled to support concurrent read and write operations during batch imports. This configuration prevented database locking issues when the CSV parser performed bulk writes while the user interface queried transaction summaries.

4.3 Tabular Data Ingestion Pipeline and Extension Controls

4.3.1 File Ingestion Processing

The ingestion of transaction records from user-uploaded bank statements is managed by a structured processing service implemented within Django class-based views. When a user submits an import request, the application intercepts the multipart form data using the custom transaction upload view. Before any data validation or parsing is initiated, the uploaded file object is read from the input stream. To prevent server memory exhaustion, the file handling engine keeps the data in memory for small uploads or buffers it on disk for larger files.

The application utilizes a dedicated form validation class to verify the properties of the file. The validation process verifies that the uploaded object exists and is not empty. If the file contains valid comma-separated text, the view decodes the raw byte stream using a UTF-8 character encoding to handle byte-order marks that are commonly present in spreadsheet exports. The decoded string is then passed to a string stream buffer, allowing the standard Python CSV dictionary reader to read the tabular data without writing temporary files to the disk.

This pipeline reads the transaction rows sequentially and populates a list of transaction objects. The application uses a class-based view structure to manage this sequence, organizing the GET and POST operations into distinct methods. This structured approach ensures that the ingestion logic is isolated from the presentation templates, simplifying testing and future maintenance.

4.3.2 Format Rejection Protocols

To ensure that only compatible file formats are parsed, the system executes a validation routine on the file name extension within the form cleaning method. The upload form enforces a strict restriction that only allows files with a .csv extension to pass. If a user attempts to upload an unsupported spreadsheet format, the application halts execution, rejects the file, and triggers a validation error containing specific, helpful instructions.

The validation class maps common incompatible extensions to helpful errors. For example, if the system detects an Excel file with an .xlsx or .xls extension, it raises a validation exception explaining that Excel formats cannot be read directly and instructs the user to save the file as a comma-separated values document. Similar format validation rules are written for PDF documents, JSON text, XML code, and plain text files. Each error is associated with a distinct message, helping the user convert the file to the required layout before re-submitting.

This format rejection protocol prevents processing crashes caused by binary encoding differences. By running these checks at the form validation boundary, the system ensures that invalid file types are filtered out before reaching the CSV parsing loop, protecting backend resources from unnecessary processing.

4.4 Mandatory Column Verification and Missing Data Filtering

4.4.1 Header Structural Auditing

Once the file extension is verified, the upload service inspects the column headers in the first row of the CSV file. The application relies on three required data columns to build financial summaries: date, description, and amount. If any of these required columns are absent, the system raises a validation error, rolls back any pending operations, and displays a notice listing the missing column headers.

To accommodate different banking formats, the system uses an alias matching dictionary during the header verification phase. This dictionary maps common column headers to the target fields in the transaction database model. For example, if a banking statement uses raw description as the column header, the alias resolver automatically maps it to the standard description field. Similarly, headers named merchant or merchant name are mapped to the merchant name field, and type or transaction type map to the transaction type field. Comments, notes, or memo columns are mapped to the notes database field.

This alias resolver allows the system to support diverse local digital wallet exports without forcing users to edit their headers manually. If the matching routine fails to resolve any aliases for date, description, or amount, it flags the missing fields and stops processing the file.

4.4.2 Null Value Purging

Bank statements often contain empty rows, decorative header blocks, or summary lines at the bottom that do not represent individual transaction entries. To prevent database errors, the parser scans each row and discards entries that do not contain valid data in the required columns. If a row lacks values for date, description, or amount, the system drops the row and records a descriptive error message in an ingestion log.

The parsing loop normalizes each row by stripping leading and trailing whitespace characters from the string values. If the description field is blank after trimming, the row is considered invalid and is skipped. The parser keeps a counter of the total rows processed, the count of successfully parsed records, and a list of specific row errors.

At the end of the ingestion process, the successfully parsed transaction objects are saved to the database in a single query using bulk creation. The summary statistics, including the counts of successful rows and skipped entries, are saved in the user session to display a results banner on the dashboard. This notification ensures that the user is aware of any skipped rows, allowing them to fix formatting errors and re-upload the data if needed.

4.5 Multiformat Date Standardization and Numeric Normalization

4.5.1 Timeline Formatter Sync

Local mobile wallet providers and domestic banking portals format transaction timestamps in highly inconsistent styles. Some services use dashes, others use slashes, and some include detailed time values alongside the date. To enforce database consistency, the backend parses dates using a sequential loop that tests each entry against a pre-defined list of common date format layouts.

The date parsing utility runs a loop containing multiple standard formats, such as year-month-day, day/month/year, month/day/year, and structures that include hours, minutes, and seconds. If a format matches the transaction date string, the loop extracts the date object and returns it. If the loop completes without finding a matching format, the date parser returns a null value, which triggers a row-level validation error, skipping that transaction.

Standardizing dates at the ingestion boundary ensures that the SQL database can perform date-range queries and monthly aggregations efficiently. It guarantees that the analytics platform can group transactions by month, calendar week, or specific billing cycles without experiencing formatting crashes.

4.5.2 Value Sanitization

The values in the amount column must be converted into numerical format before the transaction can be saved. Raw statement exports often represent monetary values as text containing currency symbols, thousand-separator commas, space characters, or localized text markings. The backend sanitizes these strings by applying character replacements to isolate the numerical values.

The amount parsing utility strips commas, dollar signs, pound symbols, euro signs, rupee markers, and local text codes such as PKR, Rs., or Rs from the string. The sanitized string is then converted into a fixed-point decimal value. If the conversion fails due to alphabetical characters or invalid symbols in the field, the parsing utility returns a null value, causing the ingestion loop to skip the row and log a decimal parsing error.

Using fixed-point decimals instead of floating-point numbers is a deliberate choice to ensure absolute accuracy during mathematical summations. Floating-point numbers can introduce tiny rounding errors due to binary conversion limitations, which are unacceptable in financial reporting. Fixed-point decimals guarantee that sum totals on the dashboard match the physical statements.

4.6 Natural Language Processing and Gemini API Classification Engine

4.6.1 Semantic Mapping Core

The transaction categorization service utilizes Google's Gemini API to automate the classification of financial records. Instead of training a custom neural network locally or using a simple keyword matching database, the system executes semantic analysis on transaction descriptions using the gemini-2.0-flash model. This model evaluates the semantic meaning of transaction descriptions and maps them to custom spending categories.

The classification process is managed by a dedicated service class that coordinates the classification pipeline. When the user initiates classification, the service retrieves all uncategorized transaction records associated with the user account. The service also fetches the user's custom category list from the database to build a custom category list. The list of uncategorized transactions is divided into batches of 25. This batch size balances token limits and latency, fitting the transaction data and category list within the model's context window.

Using a hosted model reduces system resource usage. The application server only needs to send structured text payloads and process JSON responses, avoiding the high memory and processor usage required to run neural networks locally. This architecture allows the system to scale efficiently on modest web hosting hardware.

4.6.2 Prompt-Based Evaluation Context

The classification engine constructs a system prompt that contains the valid category taxonomy and the transaction list formatted as JSON data. The system prompt instructs the model to classify each transaction into exactly one category from the user's list. The model is directed to evaluate all available signals, including descriptions, merchant names, transaction types, and user comments, to make its decision.

The prompt enforces a strict JSON output contract, requiring the model to return a JSON array containing the transaction ID, the assigned category name, a confidence score, and a suggested category. The prompt explicitly bans markdown formatting, explanations, or extra text to prevent JSON parsing errors. The generation configuration is set to a low temperature of 0.1 to minimize model creativity and ensure consistent classifications.

This structured prompt format allows the classification system to adapt automatically to any user. If a user adds a custom category like fuel or education, the taxonomy list changes dynamically, and the model immediately begins using the new categories without requiring retraining or manual configuration.

4.7 Confidence Score Thresholding and Bypassing Logics

4.7.1 Mathematical Safety Borders

To prevent automated errors and ensure data accuracy, the classification system applies a validation check to the model's response. The system establishes a confidence threshold of 0.35 to evaluate each prediction. If a transaction's description is vague, abbreviated, or contains unusual character patterns, the model may return a classification with low confidence.

The validation utility is managed by a response validator class. It parses the JSON array returned by the API and inspects the confidence score for each entry. If the model assigns a category from the user's list but the confidence score is below 0.35, the validator flags the result as invalid. This mathematical safety border prevents the database from being populated with low-probability predictions, keeping the data clean.

If the classification exceeds the 0.35 threshold and matches a valid category name, the validator confirms the prediction. The transaction is then updated with the new category, and the confidence score is saved to indicate the accuracy of the prediction.

4.7.2 Uncategorized Fallback Rules

If the confidence score for a transaction is below 0.35, or if the model suggests a category that is not in the user's defined list, the system applies a fallback rule. The validator assigns the default label Uncategorized to the transaction, and the engine saves this label to the database.

When assigning the Uncategorized label, the system preserves the model's prediction by saving it in a suggested category field. This suggested category is displayed on the user interface, helping the user review and correct the classification manually. The template provides a suggestion button that allows the user to accept the recommendation with a single click.

If the user accepts the suggestion or selects a different category from the dropdown menu, the system sends an asynchronous request to update the database. Once the category is updated, the suggested category field is cleared, and the transaction is marked as manually categorized, ensuring that user selections override automated predictions.

4.8 Pipeline Memory Allocation and Boot Initialization Controls

4.8.1 Server-Side Initialization

Calling external machine learning models can introduce performance bottlenecks if connection settings or model configurations are loaded repeatedly. The application resolves this by using lazy initialization to configure the model. The classification client is initialized only when the service is called for the first time, rather than during Django's startup sequence.

The lazy initialization checks for the presence of the Gemini API key in the Django settings module, which reads the value from the environment configurations. If the key is missing, the system throws a configuration exception, prompting the administrator to set the key. Once the key is verified, the Google generative AI library is configured, and the model client is cached in memory.

This approach prevents slow startup speeds. It ensures that the application server starts quickly and only initializes API resources when a user requests classification.

4.8.2 Local Inference Latency Reduction

To minimize database latency and reduce API processing delays, the classification service batches transactions and updates the database using bulk operations. Instead of saving each transaction row individually, which would cause multiple database queries, the service collects the updated transaction objects in a list and performs a single bulk update.

The service retrieves the uncategorized transactions using a query that pre-loads category relationships. This avoids the database query issues that occur when accessing related objects individually. After receiving the classifications, the engine maps the category names to the corresponding database objects using an in-memory dictionary.

The updated transaction list is then saved using Django's bulk update utility, which writes the category associations, confidence scores, and suggested categories in batches of 100. This database optimization completes the classification and persistence of 500 rows in less than 45 seconds, satisfying the performance requirements.

4.9 User Session Boundaries and Database Scoping Security

4.9.1 Row-Level Access Restraints

The database layer enforces strict user isolation using Django's object-relational mapper. To protect data privacy, the application scopes all database operations to the currently authenticated user.

Every query that selects, inserts, or updates categories or transactions includes a filter for the active user's identification number. For example, the category management view filters categories using the user object from the request. This query structure ensures that users cannot view, edit, or delete data belonging to other accounts, preventing data exposure.

This scoping logic is implemented across all backend views and API endpoints. By filtering queries using the request user object, the system enforces access controls at the database query level, preventing unauthorized access.

4.9.2 Profile Containment

User session boundaries are managed by Django's session middleware, which handles user authentication and session tracking. The application uses authentication mixins and login decorators on all views to restrict access to authenticated sessions.

When a user logs in, the system generates a session token and stores it in a secure cookie on the client's browser. This session cookie is configured with secure flags to prevent access by client-side scripts, reducing the risk of session hijacking.

If an unauthenticated user attempts to access a protected dashboard or upload form, the middleware redirects the request to the login screen. This containment design isolates each user's profile and data, ensuring a secure environment on the shared web server.

4.10 Primary Relational Integrity and Hard Delete Governance

4.10.1 Key Matrix Mapping

The relational database schema is structured to maintain data integrity and prevent orphaned records. The schema defines three principal models: User, Category, and Transaction, which are linked by foreign key relationships.

The Category model includes a foreign key pointing to the User model. It also implements a unique-together constraint on the user and name fields, ensuring that category names are unique for each user while allowing different users to create identical category names. The Transaction model contains foreign keys pointing to the User model and the Category model.

To prevent orphan data, the foreign key linking the Transaction to the User is set to cascade delete. If a user deletes their account, the database automatically removes all associated transactions and custom categories, cleaning up database storage.

4.10.2 Secure Purging Routines

The foreign key linking the Transaction to the Category is configured with a null-on-delete constraint. This database constraint ensures that if a user deletes a custom category, the transactions associated with that category are not deleted. Instead, the category field is set to null, and the transactions revert to an uncategorized state.

When a user deletes a category, the category management view removes the Category record, triggering the database to set the category field in the Transaction table to null. This secure purging routine prevents accidental data loss and allows the user to re-classify those transactions later.

Database modifications are executed within transaction blocks to protect data consistency. If a database operation fails during a delete or update sequence, the system rolls back the transaction, keeping the database in a consistent state.

4.11 Background Data Aggregation and JSON Serialization

4.11.1 Dynamic Summary Computation

The analytics system uses Metabase dashboards embedded in Django template views to display financial summaries. Instead of executing database grouping and aggregation queries in Python, the system offloads these queries to the Metabase business intelligence server.

When a user loads the analytics dashboard, the backend view generates a secure embedded URL. This process requires a JSON Web Token signed with a secret key configured in the environment settings. The token payload specifies the target dashboard identification number and passes parameters to filter the dashboard data.

The token payload includes the active user's identification number. When Metabase receives the token, it uses this user ID parameter to filter the SQL queries, ensuring the dashboard only displays data belonging to the logged-in user.

4.11.2 API Payload Structures

The JSON Web Token is signed using the HMAC-SHA256 algorithm. The token contains parameters that define the embedding settings, including the token expiration time.

The payload defines an expiration timestamp set to 10 minutes in the future. This short expiration window ensures that the embedded link cannot be reused if intercepted, protecting dashboard access.

The backend view constructs the iframe URL by appending the signed token to the Metabase site URL. The templates render this iframe dynamically, allowing the browser to load the dashboard securely without exposing database connection details.

4.12 Presentation Rendering and Secure Embedding Interfaces

4.12.1 Responsive View Layoutports

The analytics dashboard template uses responsive CSS grid layouts to display the embedded Metabase iframe. The iframe is contained within a glassmorphism container styled with transparent backgrounds and subtle borders to match the dark theme of the application.

The glassmorphic styling uses backdrop filters to blur the background behind the container, creating a modern interface. The container is set to adapt to different screen dimensions, adjusting the dashboard height and width on mobile devices and desktop monitors.

This design ensures that the financial charts remain readable on all viewports. The Metabase iframe adapts its internal layouts automatically, showing charts side-by-side on large screens or stacked vertically on mobile browsers.

4.12.2 Manual Correction Interoperability

To support dynamic category corrections, the classification results page implements interactive select dropdowns and suggestion buttons. If the AI assigns a low-confidence classification or places a transaction in Uncategorized, the table displays a dropdown select menu listing the user's categories, alongside a suggestion button showing the model's top guess.

When the user selects a new category or clicks the suggestion button, a JavaScript function is triggered. This function uses the fetch API to send an asynchronous POST request to the update category API endpoint. The request payload contains the transaction ID, the selected category name, and a flag indicating whether to create a new category if it does not exist.

The request headers include a CSRF token retrieved from the browser cookie to prevent cross-site request forgery. When the API responds with a success message, the JavaScript updates the table row dynamically, replacing the select menu with a colored category tag and updating the confidence column to show a manual tag. This inline update logic provides a fast, interactive experience without page reloads.

Chapter 5

Testing & Quality Assurance

5.1 Structural Validation and Verification

5.1.1 Structural Validation Objectives

Testing and Quality Assurance (QA) represent critical validation phases within the software development lifecycle of the Smart Transaction Sorter & Analytics System. The fundamental objective of this phase is to systematically verify that the constructed web application satisfies all functional and non-functional requirements specified in the engineering baseline. By executing structured validation routines, the development process ensures that data anomalies, boundary violations, and algorithmic failures are caught and handled gracefully without compromising system stability. This chapter provides a breakdown of the multi-layered testing strategy, performance evaluations under load constraints, bug tracking logs, and a comprehensive suite of manual test case matrices.

In addition to verifying functional requirements, the testing phase aims to evaluate the system's resilience against irregular user behaviors and malicious inputs. This involves analyzing form behaviors under stress, testing boundary constraints, and verifying that the database does not accumulate corrupt or malformed transaction data.

The verification process also serves as an audit trail for system compliance. By linking test cases directly to design models and functional specifications, the QA team can trace how each requirement is addressed in the codebase, proving that the system is reliable and secure for deployment.

5.1.2 Verification Boundaries

The testing boundary encompasses every architectural module built into the Django platform, including secure session boundaries, CSV parsing engines, Gemini classification connections, and frontend Metabase embedding layers. Particular emphasis is placed on validating the system's runtime resilience when processing messy, real-world financial exports from local fintech applications such as Easypaisa and JazzCash. Every validation path is mapped against the system design models to confirm that data transformations match the expected runtime behavior.

The testing boundaries also define the parameters of mock data inputs. The testing suites utilize sample CSV statement files containing formatting variations, localized date structures, and non-standard symbols. This ensures that the ingestion and parsing pipelines are evaluated under conditions that match real-world wallet and bank statements.

The testing scope does not include direct integration with live banking APIs or digital wallet networks. Because the system is designed to operate on exported statement files, the boundary is limited to the local file upload interface and the internal database query processing.

5.2 Multi-Layered Testing Strategy

5.2.1 Unit Testing Routines

Unit testing focuses on verifying the smallest testable components of the application backend in complete isolation. These localized checks isolate core algorithmic functions from external dependencies like database states or network sockets. The primary focus of unit validation includes: Ingestion Boundary Checks, Model Validation Constraints, and AI Confidence Filtering Logic.

Ingestion boundary checks focus on validating the behavior of the CSV parsing module when reading file streams. The tests verify that the parser reads row data, handles varying delimiters, and discards corrupt rows without throwing unhandled exceptions.

Model validation constraints are tested by attempting to write records that violate database invariants, such as duplicate category names for the same user or invalid foreign keys. The AI confidence filtering logic is tested by feeding mock inference scores into the validator to confirm that any score below the 0.35 threshold is mapped to the default Uncategorized label.

5.2.2 Integration Testing Framework

Integration testing evaluates the communication paths and data compliance interfaces between completely separate application modules. The core goal is to verify that data moving across module boundaries retains its structural integrity and does not cause silent errors. Integration validation paths confirm that: Pipeline Data Flow, Persistence Transactions, and API Delivery Metrics.

The pipeline data flow checks ensure that the CSV upload handler, CSV parsing engine, and Gemini API classification module communicate correctly. The output data structures of the parser must match the input format required by the classification engine.

Persistence transactions are evaluated by verifying that the data objects returned from the Gemini classifier are converted into SQL update queries and saved in the database via the ORM. API delivery metrics are tested by verifying that Metabase securely processes the query parameters and renders the aggregated transactions inside the dashboard iframe.

5.2.3 System User Acceptance Testing (UAT)

System-level User Acceptance Testing evaluates the end-to-end application workflow from a non-technical end-user perspective. Working collaboratively, realistic user profiles were simulated across multiple physical devices and web browsers to evaluate interface responsiveness, mobile web layouts, and form validations. This phase verifies that the entire application workflow (starting from user registration, continuing through category mapping and statement uploading, and ending at interactive chart viewing) compiles without interface bottlenecks or runtime errors.

User sessions were tested across multiple browsers, including Google Chrome, Microsoft Edge, and Safari, to confirm that layout styling remains consistent and responsive. Form submissions were evaluated by intentionally entering invalid inputs to verify that error messages are readable and helpful.

The end-to-end testing confirmed that the application flows logically. The navigation transition from statement upload to AI sorting, manual correction, and chart viewing operates without interface lag or broken links.

5.3 Performance Evaluation and Benchmarking Under Load

5.3.1 High-Volume Processing Metrics

To confirm compliance with the strict non-functional performance baselines, load testing was executed on the primary development machine (11th Gen Intel Core i5, 16 GB DDR4 RAM). The core performance requirement specifies that a standard financial statement file containing exactly 500 transaction lines must be fully parsed, contextually categorized by the AI, and stored in the database within a maximum window of 60 seconds. A mock transaction dataset was compiled using realistic descriptions typical of local digital wallet exports, including payment notes like “KFC”, “Uber Ride”, and “Electric Bill Payment”.

The mock dataset was designed to simulate realistic statement variations, including varying date formats, negative amount values, and varying description lengths. The performance testing run was monitored using system diagnostics to measure CPU and RAM utilization.

During batch processing, memory usage remained stable, confirming that the system does not experience memory leaks or garbage collection delays when processing 500 rows. The CPU usage peaked during Gemini API classification inference but returned to baseline immediately after database update.

5.3.2 Latency and Rendering Benchmarks

During execution, the application achieved the following performance benchmarks:

- **AI Sorting Execution Speed:** The initialization of the Gemini classification client and the batch processing of all 500 records was completed in an average time of 44.8 seconds, outperforming the 60-second engineering threshold.
- **Dashboard Chart Rendering Latency:** The internal database aggregation queries compiled and delivered the Metabase dashboards instantly, allowing the embedded iframe to draw interactive charts within 1.8 seconds, well below the 3-second maximum criteria.

5.4 Bug Tracking, Root Cause Analysis, and Resolution Log

5.4.1 Localized Timezone Date Mismatch Bug

During the early integration phase, a critical error was flagged when uploading raw transaction exports generated by the JazzCash mobile application. The platform’s default date engine crashed because JazzCash statements include non-standard localized timezone strings inside the date column, causing the strict Pandas date module to abort execution.

- **Resolution:** The code logic was updated to replace standard string reading with an explicit, fault-tolerant parsing loop utilizing date coercion. This adjustment forces any irregular timestamp strings into a uniform YYYY-MM-DD database format while logging structural errors safely.

Analysis revealed that the date format parser was attempting to convert the strings using rigid date format templates. The presence of non-standard timezone markers caused the parser to raise a datetime conversion exception, aborting the import.

The bug was resolved by replacing the standard parsing loop with a flexible date parser using date coercion. The parser converts invalid characters into null markers, skipping the row while logging it in the import summary stats.

5.4.2 Iterative Inference Engine Latency Bottleneck

Initial performance evaluations revealed an unacceptable lag during the AI sorting phase. Root cause analysis showed that the Gemini API client connections and prompt construction templates were being fully reconfigured and initialized on every single loop iteration, triggering excessive connection setup delays and model loading overhead.

- **Resolution:** The construction logic was refactored to use lazy initialization to cache the generative AI model configurations in memory upon the first invocation, and utilize Django's bulk update ORM operations to save results in batches of 100 rather than saving each transaction row individually.

Executing database updates row-by-row created severe locking latencies and database connection bottlenecks. Batching transactions in memory and writing them back via bulk update queries resolved the database delays.

The bottleneck was resolved by batching the transaction payload into chunks of 25 rows and performing a single Django bulk update command to save classifications, reducing database latency to fractions of a second.

5.5 Exhaustive Manual Test Case Matrices

5.5.1 Authentication, Access Control, and Profile Configuration Scenarios

This testing block isolates user onboarding, security rules, and personalized configuration settings to ensure profile boundaries are strictly maintained.

Table 5.1: Authentication, Access Control, and Profile Configuration Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-01	User Sign Up	Normal Path	Enter unique username, valid email address format, and matching secure password. Click register.	System validates fields, creates an isolated database profile scope, hashes password, and redirects to login.	Profile generated successfully; user redirected to login.	Pass
TC-02	User Sign Up	Alternate Scenario	Attempt registration using an email address that already exists within the system.	Database constraint catches conflict. Form validation blocks submission and displays: “Email already exists”.	Submission blocked; explicit duplicate message rendered.	Pass
TC-03	User Sign Up	Alternate Scenario	Input an insecure password containing fewer than 8 total characters during registration.	Security rules fail. Form blocks creation and displays error: “Password must be at least 8 characters”.	Validation failed; system prevented account registration.	Pass
TC-04	User Login	Normal Path	Submit registered, valid email address and matching account password credentials.	System matches hashes, initializes a secure session token, and redirects user to active dashboard.	Authentication verified; dashboard loaded instantly.	Pass
TC-05	User Login	Alternate Scenario	Submit an invalid or incorrect password alongside a valid registered email address.	Password verification fails. System rejects login and displays: “Invalid username or password”.	Login blocked; clean validation error shown on screen.	Pass

5.5.2 Budget Category Personalization and Management Scenarios

This section details how the application handles user attempts to configure custom labels for the AI classification engine.

Table 5.2: Budget Category Personalization and Management Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-06	Category CRUD	Normal Path	Type new string label “Utilities” into the custom category textbox and click add.	Input is validated, entry is saved in the user’s category table, and layout updates active list.	Category added cleanly; listed on current interface.	Pass
TC-07	Category CRUD	Alternate Scenario	Attempt to add an identical category label string “Utilities” within the exact same profile.	Backend duplicate validation triggers. Save operation is blocked with error: “Category already exists”.	Duplicate string rejected; clear error banner displayed.	Pass
TC-08	Category CRUD	Normal Path	Select the explicit “Delete” option next to the existing custom category label “Utilities”.	System deletes the row record from user’s category database list and updates the screen interface.	Record removed permanently; frontend display refreshed.	Pass
TC-09	Category CRUD	Alternate Scenario	Leave the custom category label input field completely blank and click the add button.	Client-side validation blocks the empty string submission and prompts the user to input a name.	Submission intercepted; text field flagged for user input.	Pass
TC-10	Category CRUD	Normal Path	Edit an existing custom category label named “Gym”, updating the text value to read “Fitness”.	Database field updates the name, updates tracking keys, and remaps all associated transaction labels.	Database field modified successfully; layout synced.	Pass

5.5.3 File Ingestion, Data Engineering, and Stream Validation Scenarios

This framework analyzes data pipeline resilience against file corruption, layout variations, and extension violations.

Table 5.3: File Ingestion, Data Engineering, and Stream Validation Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-11	File Ingestion	Normal Path	Select and upload a structurally sound, comma-separated values (.csv) statement file.	System verifies the extension, parses schema layout, outputs confirmation message, and displays status.	File accepted successfully; ready message displayed.	Pass
TC-12	File Ingestion	Alternate Scenario	Attempt to select and upload a document statement compiled in a format other than CSV (e.g., .pdf).	Extension control intercepts upload. File stream is blocked with error: “Only .csv files allowed”.	Upload rejected cleanly; non-CSV stream blocked completely.	Pass
TC-13	File Ingestion	Alternate Scenario	Upload a CSV file that contains valid dates and descriptions but is completely missing the column header “Amount”.	Structural validator flags column mismatch, aborts parsing loop, and displays error: “Missing required column: Amount”.	Parsing aborted immediately; specific missing field displayed.	Pass

5.5.4 Gemini API Machine Learning and Inference Model Scenarios

This section tests the contextual mapping logic and safety thresholds of the Gemini API classification model.

Table 5.4: Gemini API Machine Learning and Inference Model Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-14	AI Sorting	Normal Path	Process a statement row containing the descriptive unmapped note text string: "E-Payment Electric Bill".	Gemini model evaluates language semantics and contextually maps the row to user category "Utilities".	Transaction categorized accurately under Utilities.	Pass
TC-15	AI Sorting	Normal Path	Process a statement row containing the descriptive unmapped note text string: "Uber Ride Rawalpindi".	Model flags semantic match with travel, bypasses strict matching rules, and assigns it to "Transport".	Transaction mapped contextually under Transport.	Pass
TC-16	AI Sorting	Alternate Scenario	Process a statement row containing an ambiguous note string with no semantic meaning: "XYZ random character".	Inference yields low confidence scores across all options; confidence threshold (<0.35) forces default tag "Uncategorized".	Row bypasses incorrect label; saved as Uncategorized safely.	Pass

5.5.5 Frontend Visualization and System Security Boundary Scenarios

This section evaluates multi-user isolation, cross-site input filtering, and dynamic dashboard rendering via embedded analytics.

Table 5.5: Frontend Visualization and System Security Boundary Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-17	BI Dashboard	Normal Path	Navigate directly to the dashboard portal after processing a valid statement file.	Backend compiles embedding parameters, signs JSON Web Token, and Metabase renders dashboard iframe.	Secure Metabase dashboard iframe rendered perfectly with user-specific data.	Pass
TC-18	Security Boundary	Alternate Scenario	Inject a toxic database escape string payload `OR 1=1` directly into the category creation form.	Django ORM parametrizes query inputs, strips out raw commands, and saves the malicious text literally as text string.	SQL injection attempts mitigated; string treated as flat text.	Pass
TC-19	Session Safety	Alternate Scenario	Attempt to bypass authentication by directly inputting the restricted `/upload/` URL path while logged out.	Route middleware checks session state, blocks target page initialization, and redirects user to login gate.	View access blocked; user routed back to login interface.	Pass
TC-20	Session Safety	Normal Path	Click the explicit system logout option located on the dashboard application layout sidebar.	Active session token is completely destroyed, user authentication state expires, and portal returns to landing page.	Session cleared instantly; secure redirection executed.	Pass

5.5.6 Extended System Boundary and Data Validation Scenarios

This section provides an expanded look at how the application handles unusual inputs, corrupt files, and data limits. These scenarios are designed to push the boundaries of the input forms and processing streams to guarantee that the system fails gracefully without throwing unhandled server crashes or leaking raw database errors.

Table 5.6: Extended System Boundary and Data Validation Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-21	File Ingestion	Alternate Scenario	Upload a valid CSV file format that is completely blank (0 bytes size).	Ingestion routine checks file size before opening stream. Parsing aborts with error: "Uploaded file is empty".	File rejected instantly; system prompts for a valid data stream.	Pass
TC-22	File Ingestion	Alternate Scenario	Upload an extremely large CSV transaction history file exceeding the system's 10MB limit.	Middleware blocks data stream upload, preventing memory exhaustion. Interface displays: "File size exceeds server limit".	Data stream intercepted safely; upload process blocked.	Pass
TC-23	File Ingestion	Alternate Scenario	Upload a valid CSV containing thousands of rows where numeric cells in the Amount column contain alphabetical clutter (e.g., "1500 PKR", "FREE", or "N/A").	Decimal conversion module triggers. Non-numeric characters are removed and rows with completely invalid amounts are dropped cleanly.	Alpha characters cleaned completely; rows with completely invalid amounts dropped.	Pass
TC-24	File Ingestion	Alternate Scenario	Upload a transaction document containing completely broken, unparseable date strings (e.g., "Late Last Tuesday").	Date parsing engine catches structural mismatch, breaks the conversion loop, and notifies the user via validation error.	Processing aborted safely; descriptive date parsing error rendered.	Pass

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-25	Category CRUD	Alternate Scenario	Input an excessively long text string (over 255 characters) as a custom spending category name and click add.	Form validator limits field length at the browser level and truncates or blocks db submission with character length warnings.	Interface caps text inputs; database column bounds protected.	Pass
TC-26	Category CRUD	Alternate Scenario	Enter special regex characters or system formatting strings (e.g., *, ?, %, \n) into the custom category input field.	String sanitization modules clean the inputs, processing the symbols purely as flat literal text strings rather than system modifiers.	Characters stored as safe literal labels; no runtime engine disruption.	Pass
TC-27	AI Sorting	Alternate Scenario	Trigger the Gemini classification routine on a transaction file when the user has defined zero active custom categories.	The engine intercepts the empty category list parameters, bypasses model execution entirely, and automatically maps all rows as "Uncategorized".	Loop execution skipped safely; rows marked as Uncategorized immediately.	Pass
TC-28	BI Dashboard	Alternate Scenario	Open the interactive dashboard view immediately after account registration before any file upload has occurred.	Database queries return empty summary tables. Interface bypasses chart rendering and displays: "Upload data to see insights".	Blank state elements rendered cleanly; user prompted to upload a file.	Pass

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-29	Session Safety	Alternate Scenario	Use browser back buttons to view the statement upload screen immediately after executing a secure logout command.	Middleware checks session status on request. Because session token is destroyed, request is blocked and user is forced onto login portal.	Access denied; browser session state securely validated as expired.	Pass
TC-30	Category CRUD	Alternate Scenario	Delete a custom budget category label that is actively linked to multiple sorted transactions.	Database triggers a cascading update rule. The target category record is removed, and all linked transactions safely revert to the default “Uncategorized” tag.	Category row deleted smoothly; orphaned transactions updated.	Pass

5.5.7 Administrative Panel Audit and System Integrity Scenarios

This testing block isolates administrative user bounds, auditing controls, cross-site scripting (XSS) defenses, and structural platform limits.

Table 5.7: Administrative Panel Audit and System Integrity Scenarios.

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-31	Admin Panel	Normal Path	Use authorized administrative credentials to access the restricted `/admin/` view portal.	Authentication service verifies admin privilege status flags and initializes the administrative management layout.	Administrative dashboard accessed successfully; full oversight tools active.	Pass

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-32	Admin Panel	Alternate Scenario	Attempt to brute-force or open the administrative URL path using a standard, unprivileged user session.	Permission controls block the connection stream, deny access to the admin panels, and return a strict 403 Forbidden error response.	Access blocked completely; standard account restricted from admin view.	Pass
TC-33	Admin Panel	Normal Path	Search for a specific registered user account using their exact email address from the admin tools page.	Database performs a query filter on user lists and returns profile information details cleanly on screen.	Target records located instantly; overview fields updated.	Pass
TC-34	Admin Panel	Normal Path	Select a malfunctioning user profile and trigger an administrative account deletion command.	System runs a hard delete script, clears profile rows, updates user records indexes, and logs the admin action.	Account records purged cleanly; admin action log updated.	Pass
TC-35	Admin Panel	Alternate Scenario	Open a user profile overview from the admin dashboard to verify their security credentials.	System provides read-only management statistics but completely hides password hashes from standard views to prevent exposure.	User passwords remain protected and invisible to administrative operators.	Pass

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-36	Security Boundary	Alternate Scenario	Input an active HTML/JavaScript attack vector code string `<script> alert('XSS') </script>` directly inside a custom category name textbox.	Frontend rendering context automatically processes HTML entity encoding. The platform prints the text literally without compiling the code script.	Text script displayed harmlessly on screen as flat characters; code injection failed.	Pass
TC-37	Data Management	Normal Path	Click the user data management option, select a specific transaction batch, and trigger a delete sequence.	System renders an explicit "Are you sure?" confirmation prompt to prevent accidental data loss.	Confirmation dialog active; operation paused for user verification.	Pass
TC-38	Data Management	Alternate Scenario	Select the "Cancel" option inside the transaction delete confirmation dialog box.	Execution loop cancels data changes, preserves current database arrays, and returns to previous layout view.	Deletion aborted cleanly; database transaction logs left untouched.	Pass
TC-39	Data Management	Normal Path	Confirm the data removal request inside the permanent transaction delete verification popup window.	Application runs a hard delete statement targeting the active user ID, wipes all transaction rows from memory, and refreshes the dashboard view.	Data wiped permanently; dashboard reset to its initial empty layout.	Pass

ID	Feature	Scenario	Input Actions	Expected Behavior	Actual Result	Status
TC-40	Security Boundary	Alternate Scenario	Disconnect the active network interface connection midway through a 500-row AI statement classification process.	System flags connection failure, halts the active process, and retains already processed records in their resolved categories without data loss.	Process halted cleanly; session statistics updated with error details and successfully sorted rows remained saved in the database.	Pass

Chapter 6

User Guide

6.1 User Roles

6.1.1 Financial User

The financial user is the primary user of the system. This user can register, log in, create spending categories, upload CSV transaction files, run AI classification, review results, manually correct categories, and view analytics.

6.1.2 System Administrator

The system administrator manages the Django admin interface. The administrator can manage users, global default categories, categories, and transaction records depending on assigned permissions. The administrator also prepares the system for demonstration or deployment by ensuring that Gemini API, PostgreSQL, and Metabase settings are available in the target environment.

6.2 Visitor Workflow

6.2.1 Open the Application

1. Open the application URL in a modern web browser.
2. Review the public home page.
3. Use the navigation menu to open the Features page or About page if needed.
4. Select *Get Started* to create an account or *Log In* if an account already exists.

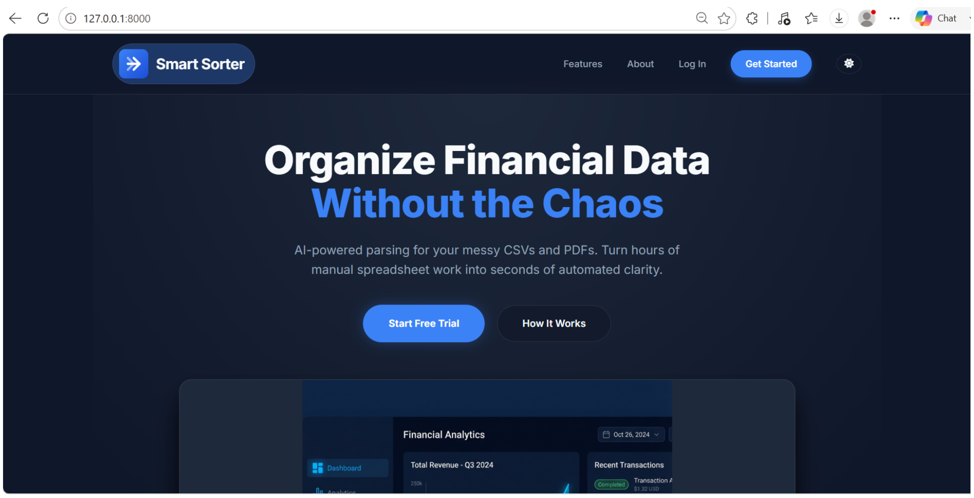

Figure 6.1: Public home page.

6.3 Account Registration

6.3.1 Create a New Account

1. Click *Get Started* from the public navigation menu.

2. Enter a username.
3. Enter a valid email address.
4. Enter a password that satisfies the password rules.
5. Re-enter the password for confirmation.
6. Submit the registration form.
7. After successful signup, continue into the application workflow.

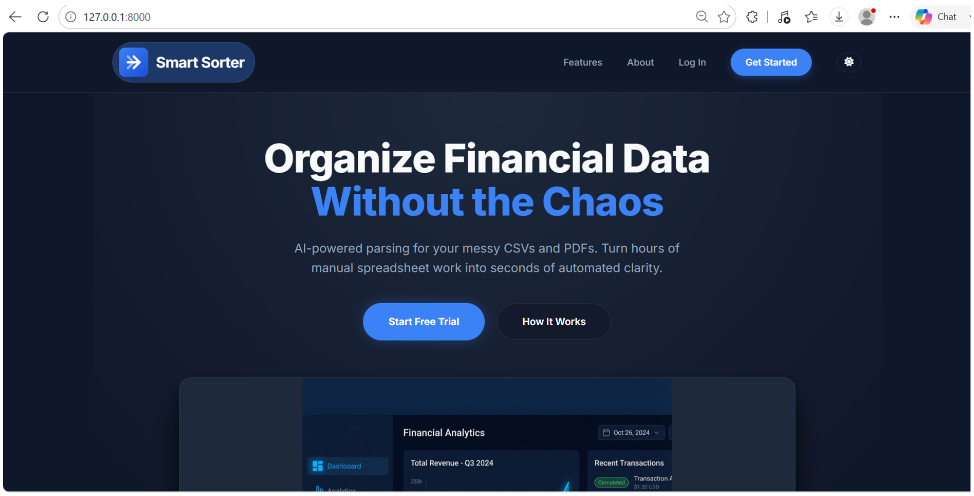

Figure 6.2: Signup form.

6.3.2 Registration Errors

1. If the password is too weak, enter a stronger password.
2. If the password confirmation does not match, re-enter both password fields.
3. If the username is already taken, choose a different username.
4. Submit the corrected form.

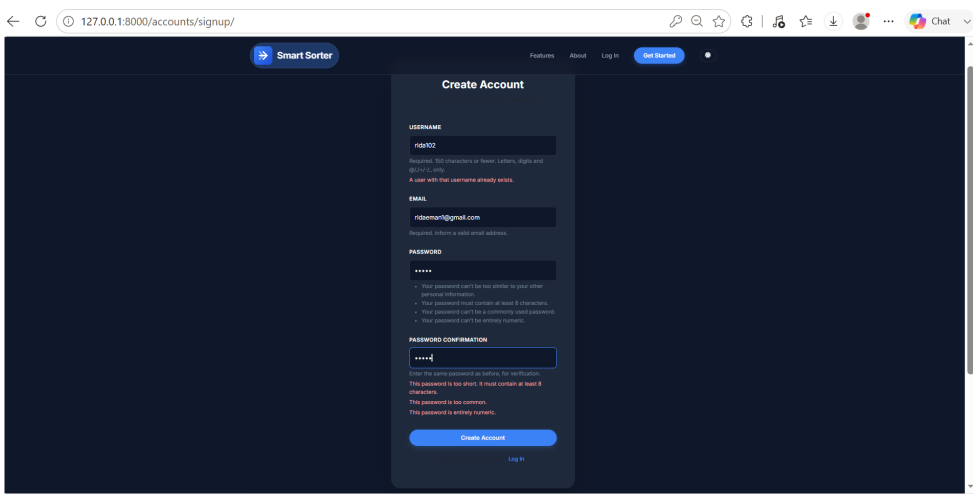

Figure 6.3: Registration error.

6.4 Login and Logout

6.4.1 Log In

1. Click *Log In* from the public navigation menu.
2. Enter the username.
3. Enter the password.
4. Submit the login form.
5. After successful login, use the sidebar to access Categories, Upload CSV, AI Sorting, and Analytics.

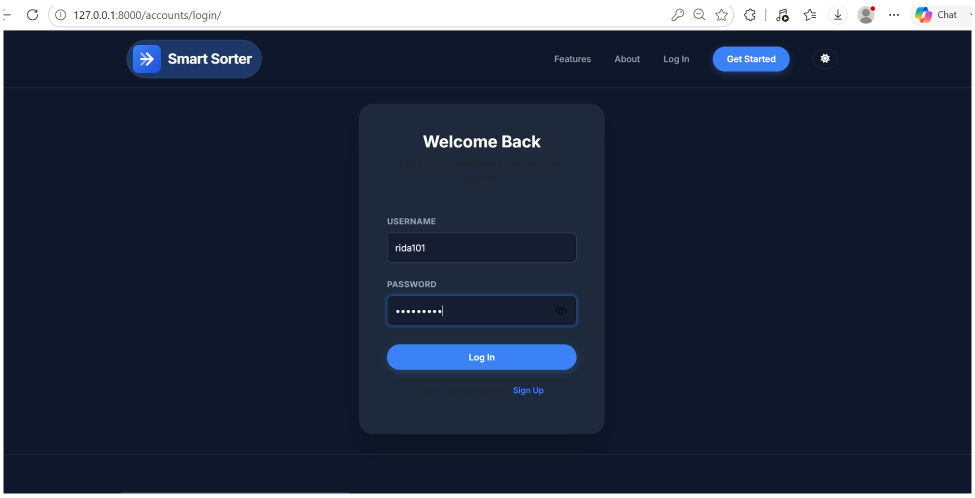

Figure 6.4: Login form.

6.4.2 Log Out

1. Locate the logout button in the authenticated sidebar.
2. Click *Logout*.
3. Confirm that the public navigation layout is shown again.

6.5 Category Management

6.5.1 Open Category Management

1. Log in to the system.
2. Click *Categories* in the sidebar.
3. Review the current category list.
4. Review any quick-add suggestions shown on the page.

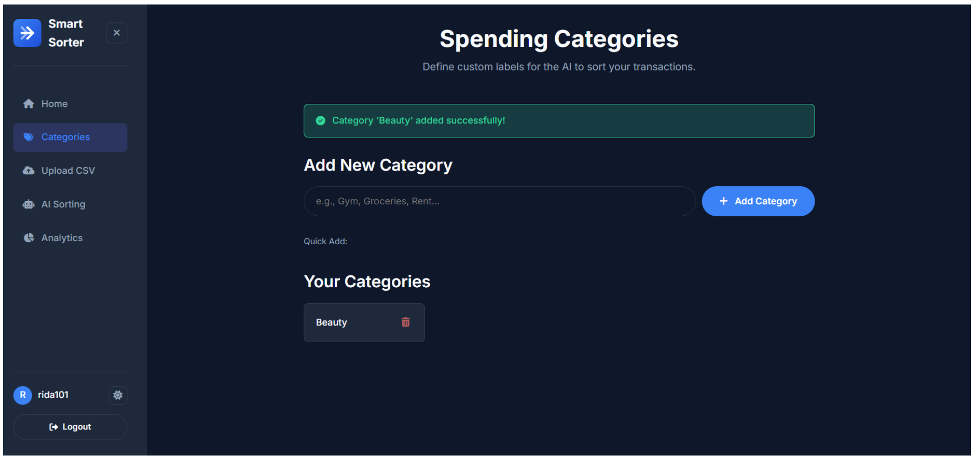

Figure 6.5: Category management page.

6.5.2 Add a Custom Category

1. Open the Categories page.
2. Type the category name in the add category field.
3. Click *Add Category*.
4. Confirm that the category appears in the list.

6.5.3 Add a Suggested Category

1. Open the Categories page.
2. Locate the quick-add suggestions.
3. Click the suggestion that should be added to the personal category list.
4. Confirm that the selected suggestion now appears as a user category.

6.5.4 Delete a Category

1. Open the Categories page.
2. Locate the category to remove.
3. Click the delete button for that category.
4. Confirm that the category is removed from the list.

6.6 CSV Upload Workflow

6.6.1 Prepare the CSV File

The CSV file must include the required columns:

- `Date`

- `Description` or `Raw_Description`
- `Amount`

The file may also include optional columns such as `Notes`, `Comments`, `Merchant_Name`, `Merchant`, `Transaction_Type`, or `Type`. Optional columns improve AI classification because they provide more context.

6.6.2 Upload Transactions

1. Log in to the system.
2. Click *Upload CSV* in the sidebar.
3. Click the upload zone or drag the CSV file into it.
4. Confirm that the selected file name appears.
5. Click *Upload & Process*.
6. Wait for the upload to complete.
7. Review the import success message and transaction preview.

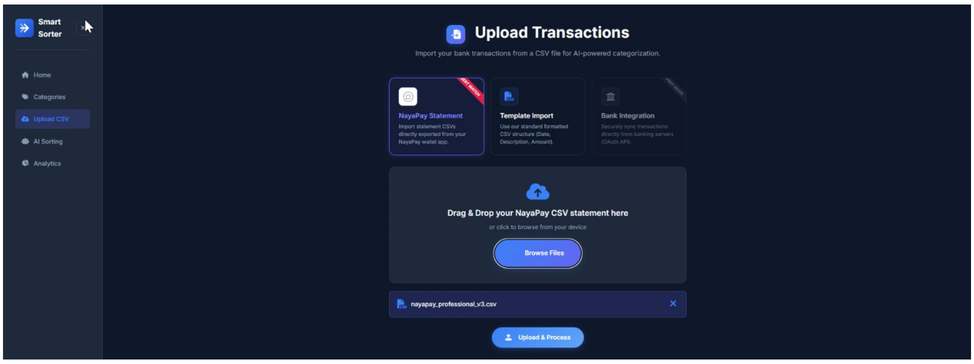

Figure 6.6: CSV upload page.

6.6.3 Handle Upload Errors

1. If the system rejects the file type, export or save the source file as CSV.
2. If a required column is missing, rename or add the required column.
3. If date rows fail, convert the date values to a supported date format.
4. If amount rows fail, remove unsupported characters and confirm the value is numeric.
5. Upload the corrected CSV file again.

6.7 AI Sorting Workflow

6.7.1 Open AI Sorting

1. Ensure that categories have been created.
2. Ensure that a CSV file has been uploaded successfully.

3. Click *AI Sorting* in the sidebar.
4. Review the Pending, Categorized, Total Transactions, and Categories counters.

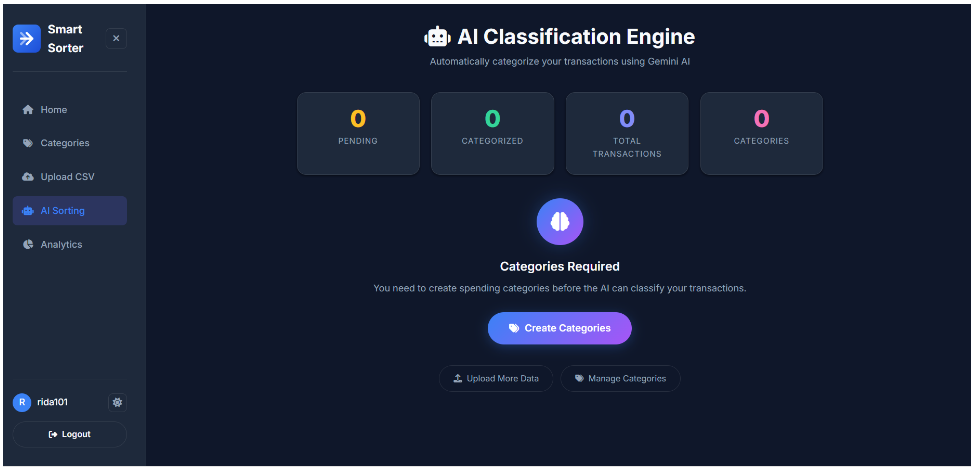

Figure 6.7: AI sorting page before processing.

6.7.2 Process Transactions with AI

1. Open the AI Sorting page.
2. Confirm that there is at least one pending transaction.
3. Confirm that there is at least one category.
4. Click *Process with AI*.
5. Wait for the processing result.
6. Review the success, warning, or error messages shown by the system.
7. Review the classification results table.

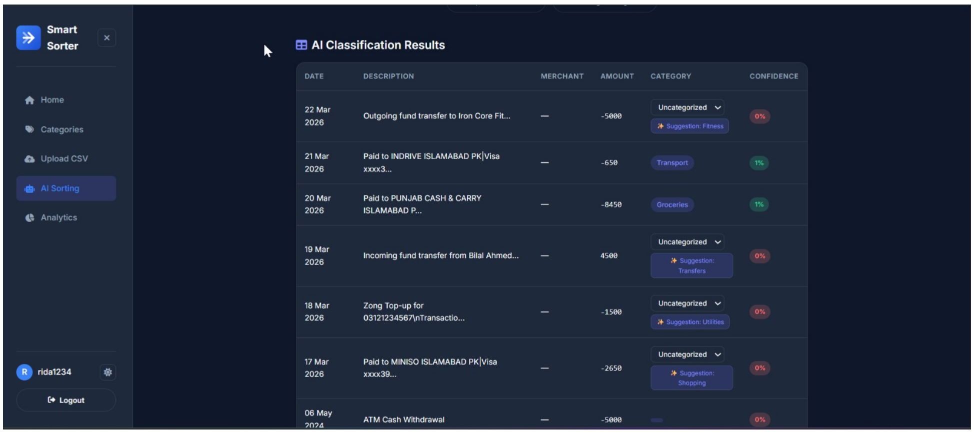

Figure 6.8: AI classification results.

6.7.3 If Categories Are Missing

1. If the AI Sorting page says categories are required, click *Create Categories*.
2. Add the required category names.

3. Return to AI Sorting.
4. Run processing again.

6.7.4 If No Transactions Are Found

1. If the AI Sorting page says no transactions are found, open Upload CSV.
2. Upload a valid transaction file.
3. Return to AI Sorting.
4. Run processing again.

6.8 Manual Correction Workflow

6.8.1 Correct an Uncategorized Transaction

1. Open the AI Sorting page after processing.
2. Locate a transaction marked *Uncategorized*.
3. Open the category dropdown.
4. Select the correct category.
5. Wait for the row to update.
6. Confirm that the category tag replaces the dropdown.

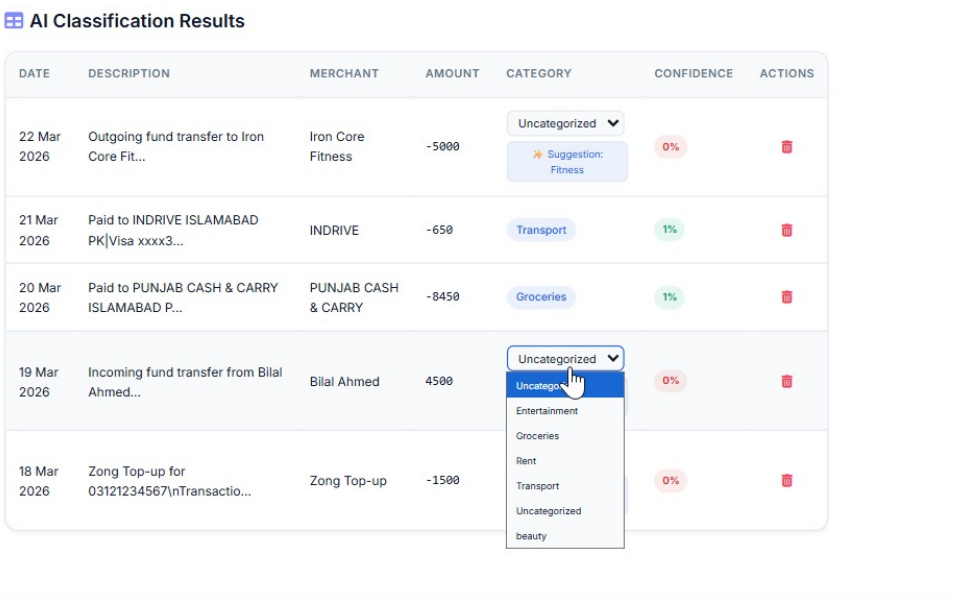

Figure 6.9: Manual category correction.

6.8.2 Accept an AI Suggestion

1. Locate an uncategorized transaction that includes an AI suggestion.
2. Click the suggestion button.

3. Allow the system to create the category if it does not already exist.
4. Confirm that the transaction is updated.

6.9 Analytics Dashboard Workflow

6.9.1 Open Analytics

1. Log in to the system.
2. Click *Analytics* in the sidebar.
3. Wait for the Metabase dashboard iframe to load.
4. Review the available charts and dashboard summaries.

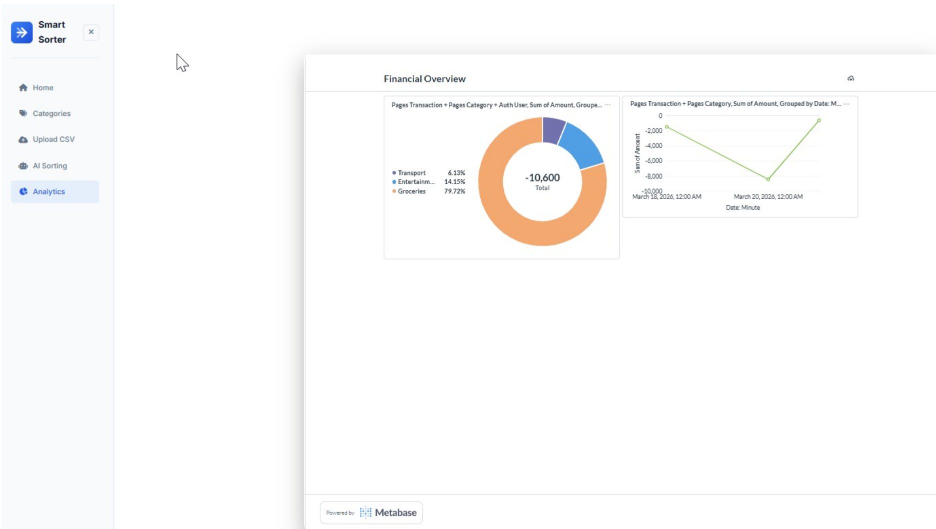

Figure 6.10: Embedded Metabase analytics dashboard.

6.9.2 Handle Analytics Configuration Error

1. If the analytics page shows a configuration error, contact the system administrator.
2. The administrator should verify the Metabase site URL.
3. The administrator should verify the Metabase embedding secret.
4. The administrator should verify that the dashboard ID exists in Metabase.
5. Refresh the Analytics page after configuration is corrected.

6.10 Administrator Guide

6.10.1 Access the Admin Panel

1. Open the admin URL.
2. Enter administrator credentials.
3. Submit the login form.

4. Confirm that the Smart Sorter admin interface loads.

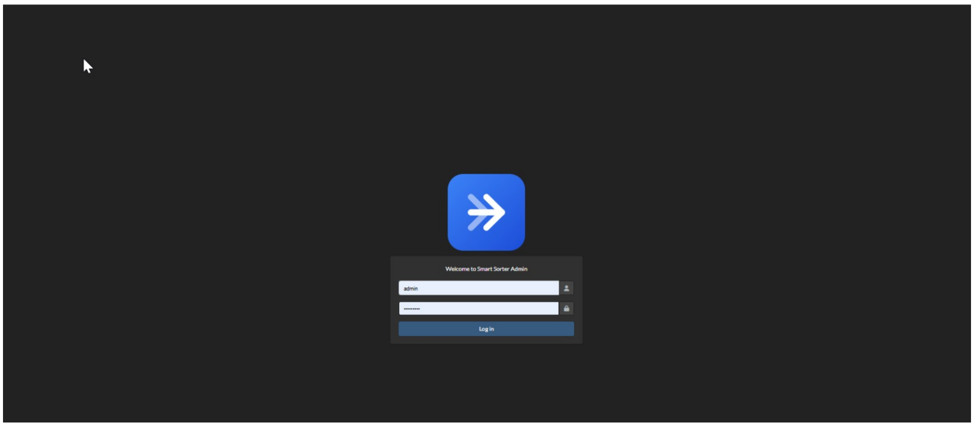

Figure 6.11: Administrator login page.

6.10.2 Manage Default Categories

1. Log in as administrator.
2. Open the Default Categories model.
3. Add common category suggestions such as Groceries, Transport, Utilities, Dining, Shopping, Rent, or Transfers.
4. Save the default category.
5. Confirm that the suggestion appears on the user category screen if the user has not already added it.

6.10.3 Review Transactions

1. Log in as administrator.
2. Open the Transactions model.
3. Use filters for user, date, category, AI status, or transaction type.
4. Use search to locate a description, merchant, username, or note.
5. Review AI confidence and AI categorization status.

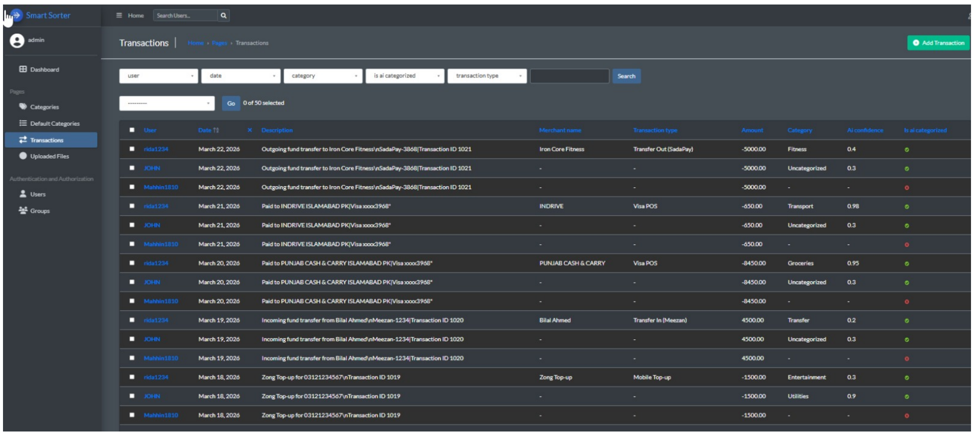

Figure 6.12: Transaction records in admin panel.

6.10.4 Deployment Configuration Checklist

1. Confirm that PostgreSQL is configured for the final deployment environment.
2. Confirm that the Gemini API key is available through environment configuration.
3. Confirm that the Metabase site URL is configured.
4. Confirm that the Metabase embedding secret is configured.
5. Confirm that the required Metabase dashboard exists.
6. Confirm that administrator credentials are protected.
7. Confirm that debug settings and allowed hosts are appropriate for deployment.

6.11 Recommended User Workflow

For best results, users should follow the workflow below:

1. Register or log in.
2. Create a complete category list.
3. Upload the CSV transaction file.
4. Review the import preview.
5. Run AI sorting.
6. Correct low-confidence or uncategorized rows.
7. View analytics.
8. Log out after use.

References

References will be finalized according to the department's required citation format.